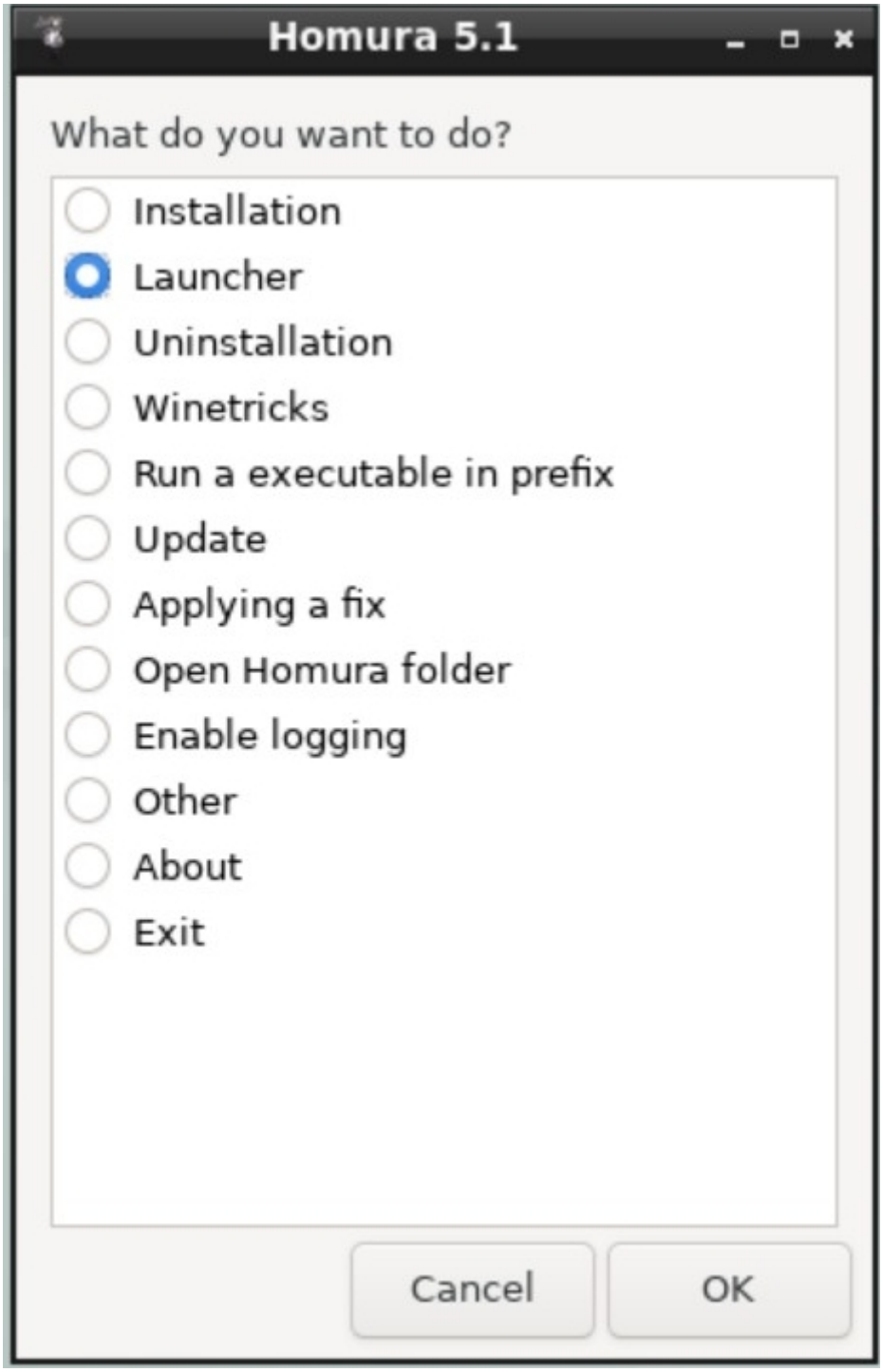
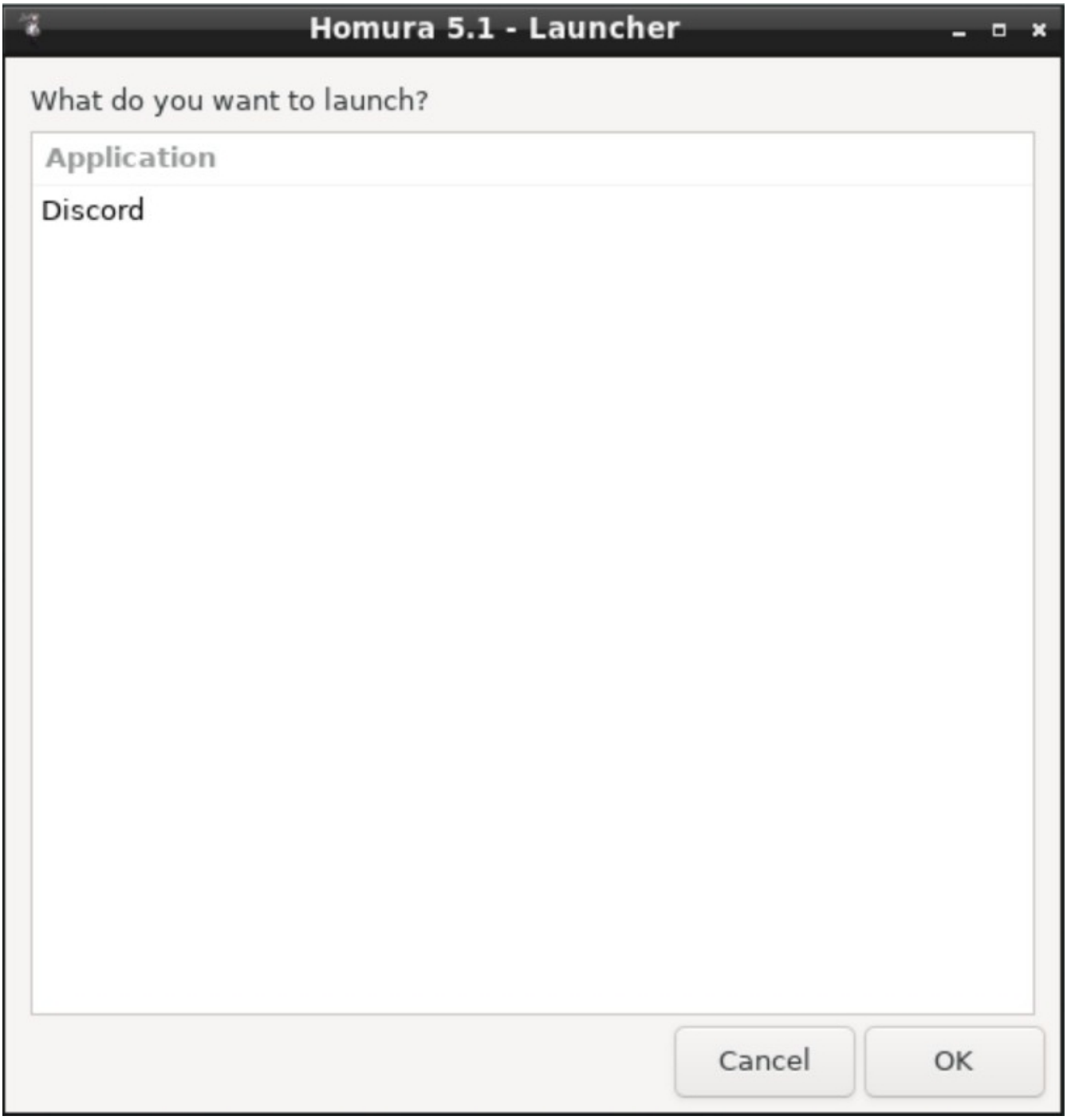
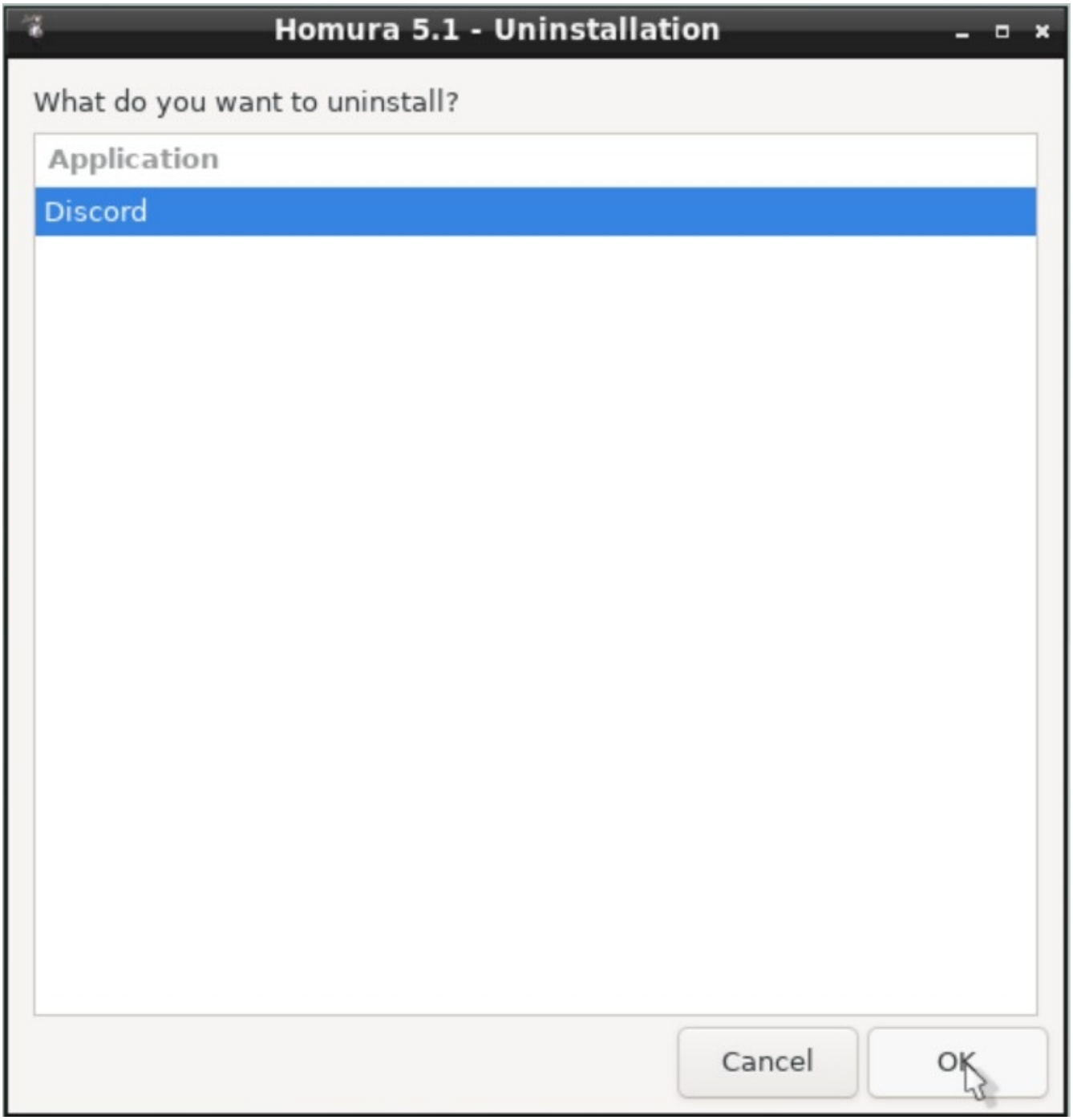


This will display a list of installed applications.



To run the new program, select it from the list, and click *OK*. To uninstall the application, select *Uninstallation* from the main screen, which will display a similar list. Select the program to be removed, and click *OK*.



13.6.3. Running Multiple Management GUIs

it is worth noting that the above solutions are not mutually exclusive. it is perfectly acceptable, even advantageous, to have both installed at the same time, as they support a different set of programs.

However, it is wise to ensure that they do not access any of the same WINE prefixes. Each of these solutions applies workarounds and makes changes to the registries based on known workarounds to existing WINE issues in order to make a given application run smoothly. Allowing both `winetricks` and Homura to access the same prefix could lead to some of these being overwritten, with the result being some or all applications do not work as expected.

13.7. WINE in Multi-User FreeBSD Installations

13.7.1. Issues with Using a Common WINE Prefix

Like most UNIX®-like operating systems, FreeBSD is designed for multiple users to be logged in and working at the same time. On the other hand, Windows® is multi-user in the sense that there can be multiple user accounts set up on one system. But the expectation is that only one will be using the physical machine (a desktop or laptop PC) at any given moment.

More recent consumer versions of Windows® have taken some steps to improve the OS in multi-user scenarios. But it is still largely structured around a single-user experience. Furthermore, the measures the WINE project has taken to create a compatible environment means, unlike FreeBSD applications (including WINE itself), it will resemble this single-user environment.

So it follows that each user will have to maintain their own set of configurations, which is potentially good. Yet it is advantageous to install applications, particularly large ones like office suites or games, only once. Two examples of reasons to do this are maintenance (software updates need only be applied once) and efficiency in storage (no duplicated files).

There are two strategies to minimize the impact of multiple WINE users in the system.

13.7.2. Installing Applications to a Common Drive

As shown in the section on WINE Configuration, WINE provides the ability to attach additional drives to a given prefix. In this way, applications can be installed to a common location, while each user will still have a prefix where individual settings may be kept (depending on the program). This is a good setup if there are relatively few applications to be shared between users, and they are programs that require few custom tweaks changes to the prefix in order to function.

The steps to make install applications in this way are as follows:

1. First, set up a shared location on the system where the files will be stored, such as /mnt/windows-drive_d/. Creating new directories is described in the mkdir(1) manual page.
2. Next, set permissions for this new directory to allow only desired users to access it. One approach to this is to create a new group such as "windows," add the desired users to that group (see the sub-section on groups in the Users and Basic Account Management section), and set to the permissions on the directory to `770` (the section on Permissions illustrates this process).
3. Finally, add the location as a drive to the user's prefix using the `winecfg` as described in the above section on WINE Configuration in this chapter.

Once complete, applications can be installed to this location, and subsequently run using the assigned drive letter (or the standard UNIX®-style directory path). However, as noted above, only one user should be running these applications (which may be accessing files within their installation directory) at the same time. Some applications may also exhibit unexpected behavior when run by a user who is not the owner, despite being a member of the group that should have full "read/write/execute" permissions for the entire directory.

13.7.3. Using a Common Installation of WINE

If, on the other hand, there are many applications to be shared, or they require specific tuning in order to work correctly, a different approach may be required. In this method, a completely separate user is created specifically for the purposes of storing the WINE prefix and all its installed applications. Individual users are then granted permission to run programs as this user using the sudo(8) command. The result is that these users can launch a WINE application as they normally would, only it will act as though launched by the newly-created user, and therefore use the centrally-maintained prefix containing both settings and programs. To accomplish this, take the following steps:

Create a new user with the following command (as `root`), which will step through the required details:

```
# adduser
```

Enter the username (e.g., *windows*) and Full name ("Microsoft Windows"). Then accept the defaults for the remainder of the questions. Next, install the `sudo` utility using binary packages with the following:

```
# pkg install sudo
```

Once installed, edit /etc/sudoers as follows:

```
# User alias specification

# define which users can run the wine/windows programs
User_Alias WINDOWS_USERS = user1,user2

# define which users can administrate (become root)
User_Alias ADMIN = user1

# Cmnd alias specification

# define which commands the WINDOWS_USERS may run
Cmnd_Alias WINDOWS = /usr/bin/wine,/usr/bin/winecfg

# Defaults
Defaults:WINDOWS_USERS env_reset
Defaults:WINDOWS_USERS env_keep += DISPLAY
Defaults:WINDOWS_USERS env_keep += XAUTHORITY
Defaults    !lecture,tty_tickets,!fqdn

# User privilege specification
root    ALL=(ALL) ALL

# Members of the admin user_alias, defined above, may gain root privileges
```

```
ADMIN ALL=(ALL) ALL

# The WINDOWS_USERS may run WINDOWS programs as user windows without a password
WINDOWS_USERS ALL = (windows) NOPASSWD: WINDOWS
```

The result of these changes is the users named in the *User_Alias* section are permitted to run the programs listed in the *Cmnd Alias* section using the resources listed in the *Defaults* section (the current display) as if they were the user listed in the final line of the file. In other words, users designates as *WINDOWS_USERS* can run the WINE and `winecfg` applications as user *windows*. As a bonus, the configuration here means they will not be required to enter the password for the *windows* user.

Next provide access to the display back to the *windows* user, as whom the WINE programs will be running:

```
% xhost +local:windows
```

This should be added to the list of commands run either at login or when the default graphical environment starts. Once all the above are complete, a user configured as one of the `WINDOW_USERS` in sudoers can run programs using the shared prefix with the following command:

```
% sudo -u windows wine program.exe
```

it is worth noting that multiple users accessing this shared environment at the same time is still risky. However, consider also that the shared environment can itself contain multiple prefixes. In this way an administrator can create a tested and verified set of programs, each with its own prefix. At the same time, one user can play a game while another works with office programs without the need for redundant software installations.

13.8. WINE on FreeBSD FAQ

The following section describes some frequently asked questions, tips/tricks, or common issues in running WINE on FreeBSD, along with their respective answers.

13.8.1. Basic Installation and Usage

13.8.1.1. How to Install 32-bit and 64-bit WINE on the Same System?

As described earlier in this section, the wine and i386-wine packages conflict with one another, and therefore cannot be installed on the same system in the normal way. However, multiple installs can be achieved using mechanisms like chroots/jails, or by building WINE from source (note this does *not* mean building the port).

13.8.1.2. Can DOS Programs Be Run on WINE?

They can, as "Console User Interface" applications as mentioned earlier in this section. However,

there is an arguably better method for running DOS software: DOSBox. On the other hand, there is little reason not to at least try it. Simply create a new prefix, install the software, and if it does not work delete the prefix.

13.8.1.3. Should the emulators/wine-devel Package/Port be Installed to Use the Development Version of WINE Instead of Stable?

Yes, installing this version will install the "development" version of WINE. As with the 32- and 64-bit versions, they cannot be installed together with the stable versions unless additional measures are taken.

Note that WINE also has a "Staging" version, which contains the most recent updates. This was at one time available as a FreeBSD port; however, it has since been removed. It can be compiled directly from source however.

13.8.2. Install Optimization

13.8.2.1. How Should Windows® Hardware (e.g., Graphics) Drivers be Handled?

Operating system drivers transfer commands between applications and hardware. WINE emulates a Windows® environment, including the drivers, which in turn use FreeBSD's native drivers for this transfer. it is not advisable to install Windows® drivers, as the WINE system is designed to use the host systems drivers. If, for example, a graphics card that benefits from dedicated drivers, install them using the standard FreeBSD methods, not Windows® installers.

13.8.2.2. Is There a way to Make Windows® Fonts Look Better?

A user on the FreeBSD forums suggests this configuration to fix out-of-the-box look of WINE fonts, which can be slightly pixelated.

According to a post in the FreeBSD Forums, adding the following to .config/fontconfig/fonts.conf will add anti-aliasing and make text more readable.

```
<?xml version="1.0"?>
<!DOCTYPE fontconfig SYSTEM "fonts.dtd>"

<fontconfig>

  <!-- antialias all fonts -->
  <match target="font">
    <edit name="antialias" mode="assign"><bool>true</bool></edit>>
    <edit name="hinting" mode="assign"><bool>true</bool></edit>>
    <edit name="hintstyle" mode="assign"><const>hintslight</const></edit>>
    <edit name="rgba" mode="assign"><const>rgb</const></edit>>
  </match>
</fontconfig>
```

13.8.2.3. Does Having Windows® Installed Elsewhere on a System Help WINE Operate?

It may, depending on the application being run. As mentioned in the section describing `winecfg`, some built-in WINE DLLs and other libraries can be overridden by providing a path to an alternate version. Provided the Windows® partition or drive is mounted to the FreeBSD system and accessible to the user, configuring some of these overrides will use native Windows® libraries and may decrease the chance of unexpected behavior.

13.8.3. Application-Specific

13.8.3.1. Where is the Best Place to see if Application X Works on WINE?

The first step in determining compatibility should be the WINE AppDB. This is a compilation of reports of programs working (or not) on all supported platforms, although (as previously mentioned), solutions for one platform are often applicable to others.

13.8.3.2. Is There Anything That Will Help Games Run Better?

Perhaps. Many Windows® games rely on DirectX, a proprietary Microsoft graphics layer. However there are projects in the open source community attempting to implement support for this technology.

The *dxvk* project, which is an attempt to implement DirectX using the FreeBSD-compatible Vulkan graphics sub-system, is one such. Although its primary target is WINE on Linux, some FreeBSD users report compiling and using dxvk.

In addition, work is under way on a wine-proton port. This will bring the work of Valve, developer of the Steam gaming platform, to FreeBSD. Proton is a distribution of WINE designed to allow many Windows® games to run on other operating systems with minimal setup.

13.8.3.3. Is There Anywhere FreeBSD WINE Users Gather to Exchange Tips and Tricks?

There are plenty of places FreeBSD users discuss issues related to WINE that can be searched for solutions:

- The FreeBSD forums, particularly the *Installation and Maintenance of Ports or Packages* or *Emulation and virtualization* forums.
- FreeBSD IRC channels including #freebsd (for general support), #freebsd-games, and others.
- The BSD World Discord server's channels including *bsd-desktop*, *bsd-gaming*, *bsd-wine*, and others.

13.8.4. Other OS Resources

There are a number of resources focused on other operating systems that may be useful for FreeBSD users:

- The WINE Wiki has a wealth of information on using WINE, much of which is applicable across many of WINE's supported operating systems.
- Similarly, the documentation available from other OS projects can also be of good value. The

WINE page on the Arch Linux Wiki is a particularly good example, although some of the "Third-party applications" (i.e., "companion applications") are obviously not available on FreeBSD.

- Finally, Codeweavers (a developer of a commercial version of WINE) is an active upstream contributor. Oftentimes answers to questions in their support forum can be of aid in troubleshooting problems with the open source version of WINE.

Part III: System Administration

The remaining chapters cover all aspects of FreeBSD system administration. Each chapter starts by describing what will be learned as a result of reading the chapter, and also details what the reader is expected to know before tackling the material.

These chapters are designed to be read as the information is needed. They do not need to be read in any particular order, nor must all of them be read before beginning to use FreeBSD.

Chapter 14. Configuration, Services, Logging and Power Management

14.1. Synopsis

One of the important aspects of FreeBSD is proper system configuration. This chapter explains much of the FreeBSD configuration process, including some of the parameters which can be set to tune a FreeBSD system.

Before reading this chapter:

- Understand UNIX® and FreeBSD basics (FreeBSD Basics).

Read this chapter to learn:

- How to use the various configuration files in /etc.
- The basics of rc.conf configuration and /usr/local/etc/rc.d startup scripts.
- How to tune FreeBSD using sysctl(8) variables.
- How to configure the power management in FreeBSD.

14.2. Configuration Files

FreeBSD maintains a clear separation between the base system and third party applications and therefore this affects where the configuration files of these applications are located.

FreeBSD base system configuration is located at the /etc directory, and the /usr/local/etc directory contains all the configuration files of the applications installed on the system through the ports collection and packages.

The kernel state configuration is located in /etc/sysctl.conf. In the section The sysctl utility, the operation of sysctl(8) will be explained in more detail.

For more information about the FreeBSD file system structure refer to hier(7).

As a general rule, configuration files do not use a standard on what syntax they must follow. Although it is true that the # character is normally used to comment a line and that each line has a configuration variable.



Some applications like pkg(8) are starting to use the Universal Configuration Language (UCL).

14.2.1. The /etc directory

The /etc directory contains all of the FreeBSD base system configuration files that are responsible for configuring FreeBSD.



Extreme caution must be taken when modifying files in the /etc directory; misconfiguration could make FreeBSD unbootable or malfunction.

/etc	System configuration files and scripts.
/etc/defaults	Default system configuration files, see rc(8) for more information.
/etc/fstab	fstab(5) contains descriptive information about the various file systems.
/etc/mail	Extra sendmail(8) configuration and other MTA configuration files.
/etc/mtree	mtree configuration files, see mtree(8) for more information.
/etc/pam.d	Configuration files for the Pluggable Authentication Modules (PAM) library.
/etc/periodic	Scripts that are run daily, weekly, and monthly, via cron(8), see periodic(8) for more information.
/etc/rc.d	System and daemon startup/control scripts, see rc(8) for more information.
/etc/rc.conf	Contains descriptive information about the local host name, configuration details for any potential network interfaces and which services should be started up at system initial boot time. More information in Managing System-Specific Configuration
/etc/security	OpenBSM audit configuration files, see audit(8) for more information.
/etc/ppp	ppp configuration files, see ppp(8) for more information.
/etc/ssh	OpenSSH configuration files, see ssh(1) for more information.
/etc/ssl	OpenSSL configuration files.
/etc/sysctl.conf	Contains settings for the kernel. More information in The sysctl utility

14.2.2. The sysctl utility

The sysctl(8) utility is used to make changes to a running FreeBSD system.

The sysctl(8) utility retrieves kernel state and allows processes with appropriate privilege to set kernel state. The state to be retrieved or set is described using a "Management Information Base" ("MIB") style name, described as a dotted set of components.

Table 24. Management Information Base

sysctl	"Magic" numbers
kern	Kernel functions and features
vm	Virtual memory
vfs	Filesystem
net	Network
debug	Debugging parameters
hw	Hardware
machdep	Machine dependent
user	Userland
p1003_1b	POSIX 1003.1B

At its core, sysctl(8) serves two functions: to read and to modify system settings.

To view all readable variables:

```
% sysctl -a
```

The output should be similar to the following:

```
kern.ostype: FreeBSD
...
vm.swap_enabled: 1
vm.overcommit: 0
vm.domain.0.pidctrl.kdd: 8
vm.domain.0.pidctrl.kid: 4
vm.domain.0.pidctrl.kpd: 3
...
vfs.zfs.sync_pass_rewrite: 2
vfs.zfs.sync_pass_dont_compress: 8
vfs.zfs.sync_pass_deferred_free: 2
```

To read a particular variable, specify its name:

```
% sysctl kern.maxproc
```

The output should be similar to the following:

```
kern.maxproc: 1044
```

The Management Information Base (MIB) is hierarchical and hence, specifying a prefix prints all

the nodes hanging from it:

```
% sysctl net
```

The output should be similar to the following:

```
net.local.stream.recvspace: 8192
net.local.stream.sendspace: 8192
net.local.dgram.recvspace: 16384
net.local.dgram.maxdgram: 2048
net.local.seqpacket.recvspace: 8192
net.local.seqpacket.maxseqpacket: 8192
net.local.sockcount: 60
net.local.taskcount: 25
net.local.recycled: 0
net.local.deferred: 0
net.local.inflight: 0
net.inet.ip.portrange.randomtime: 1
net.inet.ip.portrange.randomcps: 9999
[...]
```

To set a particular variable, use the *variable=value* syntax:

```
# sysctl kern.maxfiles=5000
```

The output should be similar to the following:

```
kern.maxfiles: 2088 -> 5000
```



To keep the configuration after a reboot it is necessary to add these variables to the /etc/sysctl.conf file as explained below.

14.2.3. The /etc/sysctl.conf file

The configuration file for sysctl(8), /etc/sysctl.conf, looks much like /etc/rc.conf.

Values are set using a `variable=value` syntax.



The specified values are set after the system goes into multi-user mode. Not all variables are settable in this mode.

For example, to turn off logging of fatal signal exits and prevent users from seeing processes started by other users, the following tunables can be set in /etc/sysctl.conf:

```
# Do not log fatal signal exits (e.g., sig 11)
```

```
kern.logsigexit=0

# Prevent users from seeing information about processes that
# are being run under another UID.
security.bsd.see_other_uids=0
```

To obtain more information about what function a particular sysctl has, the following command can be executed:

```
% sysctl -d kern.dfldsiz
```

The output should be similar to the following:

```
kern.dfldsiz: Initial data size limit
```

14.2.4. Managing System-Specific Configuration

The principal location for system configuration information is /etc/rc.conf.

This file contains a wide range of configuration information and it is read at system startup to configure the system. It provides the configuration information for the rc* files.

The entries in /etc/rc.conf override the default settings in /etc/defaults/rc.conf.



The file /etc/defaults/rc.conf containing the default settings should not be edited. Instead, all system-specific changes should be made to /etc/rc.conf.

A number of strategies may be applied in clustered applications to separate site-wide configuration from system-specific configuration in order to reduce administration overhead.

The recommended approach is to place system-specific configuration into /etc/rc.conf.local.

For example, these entries in /etc/rc.conf apply to all systems:

```
sshd_enable="YES"
keyrate="fast"
defaultrouter="10.1.1.254"
```

Whereas these entries in /etc/rc.conf.local apply to this system only:

```
hostname="node1.example.org"
ifconfig_fxp0="inet 10.1.1.1/8"
```

Distribute /etc/rc.conf to every system using an application such as rsync or puppet, while /etc/rc.conf.local remains unique.

Upgrading the system will not overwrite /etc/rc.conf, so system configuration information will not be lost.



Both /etc/rc.conf and /etc/rc.conf.local are parsed by sh(1). This allows system operators to create complex configuration scenarios. Refer to rc.conf(5) for further information on this topic.

14.3. Managing Services in FreeBSD

FreeBSD uses the rc(8) system of startup scripts during system initialization and for managing services.

The scripts listed in /etc/rc.d provide basic services which can be controlled with the `start`, `stop`, and `restart` options to service(8).

A basic script may look similar to the following:

```
#!/bin/sh
#
# PROVIDE: utility
# REQUIRE: DAEMON
# KEYWORD: shutdown

. /etc/rc.subr

name=utility
rcvar=utility_enable

command="/usr/local/sbin/utility"

load_rc_config $name

#
# DO NOT CHANGE THESE DEFAULT VALUES HERE
# SET THEM IN THE /etc/rc.conf FILE
#
utility_enable=${utility_enable-"NO"}
pidfile=${utility_pidfile-"/var/run/utility.pid"}

run_rc_command "$1"
```

Refer to this article for instructions on how to create custom rc(8) scripts.

14.3.1. Starting Services

Many users install third party software on FreeBSD from the Ports Collection and require the installed services to be started upon system initialization.

Services, such as security/openssh-portable or www/nginx are just two of the many software packages which may be started during system initialization. This section explains the procedures available for starting services.

Since the rc(8) system is primarily intended to start and stop services at system startup and shutdown time, the `start`, `stop` and `restart` options will only perform their action if the appropriate /etc/rc.conf variable is set.

So the first step to start a service, like for example www/nginx is to add it to /etc/rc.conf by executing the following command:

```
# sysrc nginx_enable="YES"
```

Then nginx can be started executing the following command:

```
# service nginx start
```



To `start`, `stop` or `restart` a service regardless of the settings in /etc/rc.conf, these commands should be prefixed with "one". For instance, to start www/nginx regardless of the current /etc/rc.conf setting, execute the following command:

```
# service nginx onestart
```

It is also possible to put a service automatically into a jail, see the corresponding Service Jails explanation.

14.3.2. Status of a Service

To determine if a service is running, use the `status` subcommand.

For example, to verify that www/nginx is running:

```
# service nginx status
```

The output should be similar to the following:

```
nginx is running as pid 27871.
```

14.3.3. Reload a Service

In some cases, it is also possible to `reload` a service. This attempts to send a signal to an individual service, forcing the service to reload its configuration files.

In most cases, this means sending the service a `SIGHUP` signal.

Not all services support this feature.

The rc(8) system is used for network services and it also contributes to most of the system initialization. For instance, when the /etc/rc.d/bgfsck script is executed, it prints out the following message:

```
Starting background file system checks in 60 seconds.
```

This script is used for background file system checks, which occur only during system initialization.

Many system services depend on other services to function properly. For example, yp(8) and other RPC-based services may fail to start until after the rpcbind(8) service has started.

Additional information can be found in rc(8) and rc.subr(8).

14.3.4. Using Services to Start Services

Other services can be started using inetd(8). Working with inetd(8) and its configuration is described in depth in "The inetd Super-Server".

In some cases, it may make more sense to use cron(8) to start system services. This approach has a number of advantages as cron(8) runs these processes as the owner of the crontab(5). This allows regular users to start and maintain their own applications.

The `@reboot` feature of cron(8), may be used in place of the time specification. This causes the job to run when cron(8) is started, normally during system initialization.

14.4. Cron and Periodic

Scheduling tasks to run at a certain day or time is a very common task on FreeBSD. The tool in charge of performing this task is cron(8).

In addition to tasks that can be scheduled by the user via cron(8), FreeBSD performs routine background tasks managed by periodic(8).

14.4.1. Cron

The cron(8) utility runs in the background and regularly checks /etc/crontab for tasks to execute and searches /var/cron/tabs for custom crontab files.

These files are used to schedule tasks which cron runs at the specified times.

Each entry in a crontab defines a task to run and is known as a *cron job*.

Two different types of configuration files are used: the system crontab, which should not be modified, and user crontabs, which can be created and edited as needed. The format used by these files is documented in crontab(5). The format of the system crontab, /etc/crontab includes a `who` column which does not exist in user crontabs. In the system crontab, cron runs the command as the user specified in this column. In a user crontab, all commands run as the user who created the

crontab.

User crontabs allow individual users to schedule their own tasks. The `root` user can also have a user crontab which can be used to schedule tasks that do not exist in the system crontab.

Here is a sample entry from the system crontab, /etc/crontab:

```
# /etc/crontab - root's crontab for FreeBSD
#
①
#
SHELL=/bin/sh
PATH=/sbin:/bin:/usr/sbin:/usr/bin:/usr/local/sbin:/usr/local/bin ②
#
#minute hour    mday    month   wday    who     command ③
#
# Save some entropy so that /dev/random can re-seed on boot.
*/11    *       *       *       *       operator /usr/libexec/save-entropy ④
#
# Rotate log files every hour, if necessary.
0       *       *       *       *       root    newsyslog
#
# Perform daily/weekly/monthly maintenance.
1       3       *       *       *       root    periodic daily
15      4       *       *       6       root    periodic weekly
30      5       1       *       *       root    periodic monthly
#
# Adjust the time zone if the CMOS clock keeps local time, as opposed to
# UTC time.  See adjkerntz(8) for details.
1,31    0-5     *       *       *       root    adjkerntz -a
```

① Lines that begin with the `#` character are comments. A comment can be placed in the file as a reminder of what and why a desired action is performed. Comments cannot be on the same line as a command or else they will be interpreted as part of the command; they must be on a new line. Blank lines are ignored.

② The equals (`=`) character is used to define any environment settings. In this example, it is used to define the `SHELL` and `PATH`. If the `SHELL` is omitted, cron will use the default POSIX® shell. If the `PATH` is omitted, the full path must be given to the command or script to run.

③ This line defines the seven fields used in a system crontab: `minute`, `hour`, `mday`, `month`, `wday`, `who`, and `command`. The `minute` field is the time in minutes when the specified command will be run, the `hour` is the hour when the specified command will be run, the `mday` is the day of the month, `month` is the month, and `wday` is the day of the week. These fields must be numeric values, representing the twenty-four hour clock, or a `*`, representing all values for that field. The `who` field only exists in the system crontab and specifies which user the command should be run as. The last field is the command to be executed.

④ This entry defines the values for this cron job. The `*/11`, followed by several more `*` characters, specifies that `/usr/libexec/save-entropy` is invoked by `operator` every eleven minutes of every hour, of every day and day of the week, of every month. Commands can include any number of

switches. However, commands which extend to multiple lines need to be broken with the backslash "\" continuation character.

14.4.2. Creating a User Crontab

To create a user crontab, invoke `crontab` in editor mode:

```
% crontab -e
```

This will open the user's crontab using the default text editor. The first time a user runs this command, it will open an empty file. Once a user creates a crontab, this command will open that file for editing.

It is useful to add these lines to the top of the crontab file in order to set the environment variables and to remember the meanings of the fields in the crontab:

```
SHELL=/bin/sh
PATH=/sbin:/bin:/usr/sbin:/usr/bin:/usr/local/sbin:/usr/local/bin
# Order of crontab fields
# minute hour mday month wday command
```

Then add a line for each command or script to run, specifying the time to run the command. This example runs the specified custom shell script every day at two in the afternoon. Since the path to the script is not specified in `PATH`, the full path to the script is given:

```
0 14 * * * /home/user/bin/mycustomscript.sh
```



Before using a custom script, make sure it is executable and test it with the limited set of environment variables set by cron. To replicate the environment that would be used to run the above cron entry, use:

```
env -i SHELL=/bin/sh PATH=/etc:/bin:/sbin:/usr/bin:/usr/sbin
HOME=/home/user LOGNAME=user /home/user/bin/mycustomscript.sh
```

The environment set by cron is discussed in crontab(5). Checking that scripts operate correctly in a cron environment is especially important if they include any commands that delete files using wildcards.

When finished editing the crontab, save the file. It will automatically be installed, and cron will read the crontab and run its cron jobs at their specified times. To list the cron jobs in a crontab, use this command:

```
% crontab -l
```

The output should be similar to the following:

```
0 14 * * * /home/user/bin/mycustomscript.sh
```

To remove all of the cron jobs in a user crontab:

```
% crontab -r
```

The output should be similar to the following:

```
remove crontab for user? y
```

14.4.3. Periodic

FreeBSD provides a set of system management scripts to check status of various subsystems, perform security-related checks, rotate log files, etc. These scripts are run on a periodic basis: daily. weekly, or monthly. The management of these tasks is performed by periodic(8) and its configuration resides in periodic.conf(5). The periodic tasks are initiated by entries in the system crontab, shown above.

Scripts executed by periodic(8) are located in /etc/periodic/ for base utilities and in /usr/local/etc/periodic/ for third-party software.

They are organized in 4 subdirectories, daily, weekly, monthly and security.

14.4.4. Enable or Disable Periodic Tasks

FreeBSD has some scripts enabled by default to run periodically.

To enable or disable a task, the first step is to edit /etc/periodic.conf executing the following command:

```
# ee /etc/periodic.conf
```

And then to enable, for example, `daily_status_zfs_enable` put the following content in the file:

```
daily_status_zfs_enable="YES"
```

To disable a task that is active by default, all that needs to be done is to change `YES` to `NO`.

14.4.5. Configuring the Output of Periodic Tasks

In /etc/periodic.conf the variables `daily_output`, `weekly_output` and `monthly_output` specifies where to send the results of the script execution.

By default the output of the periodic scripts are emailed to root, and therefore it is best to read root's mail or alias root to a mailbox that is monitored.

To send the results to another email or to other emails, add the email addresses separated by spaces to /etc/periodic.conf:

```
daily_output="email1@example.com email2@example.com"
weekly_output="email1@example.com email2@example.com"
monthly_output="email1@example.com email2@example.com"
```

To log periodic output instead of receiving it as email, add the following lines to /etc/periodic.conf. newsyslog(8) will rotate these files at the appropriate times:

```
daily_output=/var/log/daily.log
weekly_output=/var/log/weekly.log
monthly_output=/var/log/monthly.log
```

14.5. Configuring System Logging

Generating and reading system logs is an important aspect of system administration. The information in system logs can be used to detect hardware and software issues as well as application and system configuration errors. This information also plays an important role in security auditing and incident response. Most system daemons and applications will generate log entries.

FreeBSD provides a system logger, syslogd(8), to manage logging. By default, syslogd is enabled and started when the system boots.

This section describes how to configure the FreeBSD system logger for both local and remote logging and how to perform log rotation and log management.

14.5.1. Configuring Local Logging

The configuration file, /etc/syslog.conf, controls what syslogd does with log entries as they are received. There are several parameters to control the handling of incoming events. The *facility* describes which subsystem generated the message, such as the kernel or a daemon, and the *level* describes the severity of the event that occurred. This makes it possible to configure if and where a log message is logged, depending on the facility and level. It is also possible to take action depending on the application that sent the message, and in the case of remote logging, the hostname of the machine generating the logging event.

This configuration file contains one line per action, where the syntax for each line is a selector field followed by an action field. The syntax of the selector field is *facility.level* which will match log messages from *facility* at level *level* or higher. It is also possible to add an optional comparison flag before the level to specify more precisely what is logged. Multiple selector fields can be used for the same action, and are separated with a semicolon (`;`). Using `*` will match everything. The action field denotes where to send the log message, such as to a file or remote log host.

As an example, here is the default /etc/syslog.conf from FreeBSD:

```
#       Spaces ARE valid field separators in this file. However,
#       other *nix-like systems still insist on using tabs as field
#       separators. If you are sharing this file between systems, you
#       may want to use only tabs as field separators here.
#       Consult the syslog.conf(5) manpage.
*.err;kern.warning;auth.notice;mail.crit                /dev/console ①
*.notice;authpriv.none;kern.debug;lpr.info;mail.crit;news.err   /var/log/messages
security.*                                      /var/log/security
auth.info;authpriv.info                         /var/log/auth.log
mail.info                                       /var/log/maillog ②
cron.*                                          /var/log/cron
!-devd
*.=debug                                        /var/log/debug.log ③
*.emerg                                         *
daemon.info                                     /var/log/daemon.log
# uncomment this to log all writes to /dev/console to /var/log/console.log
# touch /var/log/console.log and chmod it to mode 600 before it will work
#console.info                                   /var/log/console.log
# uncomment this to enable logging of all log messages to /var/log/all.log
# touch /var/log/all.log and chmod it to mode 600 before it will work
#*.*                                            /var/log/all.log
# uncomment this to enable logging to a remote loghost named loghost
#*.*                                            @loghost
# uncomment these if you're running inn
# news.crit                                     /var/log/news/news.crit
# news.err                                      /var/log/news/news.err
# news.notice                                   /var/log/news/news.notice
# Uncomment this if you wish to see messages produced by devd
# !devd
# *.>=notice                                    /var/log/devd.log ④
!*
include                                         /etc/syslog.d
include                                         /usr/local/etc/syslog.d
```

① Matches all messages with a level of `err` or higher, as well as `kern.warning`, `auth.notice` and `mail.crit`, and sends these log messages to the console (/dev/console).

② Matches all messages from the `mail` facility at level `info` or above and logs the messages to /var/log/maillog.

③ Uses a comparison flag (`=`) to only match messages at level `debug` and logs them to /var/log/debug.log.

④ Is an example usage of a program specification. This makes the rules following it only valid for the specified program. In this case, only the messages generated by devd(8) are logged to /var/log/devd.log.

For more information about /etc/syslog.conf, its syntax, and more advanced usage examples, see syslog.conf(5).

14.5.2. Logging Facilities

A facility describes the part of the system generating the message. Facilities are a way of separating the different messages so that it is easier for the user to consult the logs.

Table 25. syslog facilities

Name	Description
auth	The authorization system: login(1), su(1), getty(8), etc.
authpriv	The same as auth, but logged to a file readable only by root.
console	Messages written to /dev/console by the kernel console output driver.
cron	Messages written by the cron(8) daemon.
daemon	System daemons, such as routed(8), that are not provided for explicitly by other facilities.
ftp	The file transfer protocol daemons: ftpd(8), tftpd(8).
kern	Messages generated by the kernel. These cannot be generated by any user processes.
lpr	The line printer spooling system: lpr(1), lpc(8), lpd(8), etc.
mail	The mail system.
mark	This facility adds a record every 20 minutes.
news	The network news system.
ntp	The network time protocol system.
security	Security subsystems, such as ipfw(4).
syslog	Messages generated internally by syslogd(8).
user	Messages generated by random user processes. **This is the default facility identifier if none is specified**.
uucp	The Unix-to-Unix Copy system. An ancient protocol. Really weird to see messages from this facility.
local0 through local7	Reserved for local use.

14.5.3. Logging Levels

The level describes the severity of the message, and is a keyword from the following ordered list (higher to lower):

Table 26. syslog levels

Name	Description
emerg	A panic condition. This is normally broadcast to all users.
alert	A condition that should be corrected immediately, such as a corrupted system database.
crit	Critical conditions, e.g., hard device errors.
err	Errors.
warning	Warning messages.
notice	Conditions that are not error conditions, but should possibly be handled specially.
info	Informational messages.
debug	Messages that contain information normally of use only when debugging a program.
none	This special level disables a particular facility.

14.5.4. Read Log Messages

By default FreeBSD log files use the format rfc3164, also known as The BSD syslog Protocol. Learn more about other formats and how to use them at syslog(8).

Typically the logs have the following syntax:

```
date time hostname program[pid]: the message
```

The output of the /var/log/cron file will be used as an example:

```
[...]
Jul 16 12:40:00 FreeBSD /usr/sbin/cron[81519]: (root) CMD (/usr/libexec/atrun)
Jul 16 12:44:00 FreeBSD /usr/sbin/cron[83072]: (operator) CMD (/usr/libexec/save-
entropy)
[...]
```

Verbose logging, so the facility and the level on each message will be added, can be enabled in syslog(8) by running the following command:

```
# sysrc syslogd_flags="-vv"
```

Once the function is activated, the facility and the level will be displayed in the log as shown in the following example:

```
[...]
Jul 16 17:40:00 <cron.info> FreeBSD /usr/sbin/cron[1016]: (root) CMD
(/usr/libexec/atrun)
Jul 16 17:44:00 <cron.info> FreeBSD /usr/sbin/cron[1030]: (operator) CMD
(/usr/libexec/save-entropy)
[...]
```

14.5.5. Log Management and Rotation

Log files can grow quickly, taking up disk space and making it more difficult to locate useful information.

In FreeBSD, newsyslog(8) is used to manage log files and attempt to mitigate this.

This built-in program periodically rotates and compresses log files, and optionally creates missing log files and signals programs when log files are moved.



Since newsyslog is run from cron(8), it cannot rotate files more often than it is scheduled to run from cron(8). In the default configuration, it runs every hour.

Here is the default configuration in FreeBSD, more information in newsyslog.conf(5):

```
# configuration file for newsyslog
#
# Entries which do not specify the '/pid_file' field will cause the
# syslogd process to be signalled when that log file is rotated.  This
# action is only appropriate for log files which are written to by the
# syslogd process (ie, files listed in /etc/syslog.conf).  If there
# is no process which needs to be signalled when a given log file is
# rotated, then the entry for that file should include the 'N' flag.
#
# Note: some sites will want to select more restrictive protections than the
# defaults.  In particular, it may be desirable to switch many of the 644
# entries to 640 or 600.  For example, some sites will consider the
# contents of maillog, messages, and lpd-errs to be confidential.  In the
# future, these defaults may change to more conservative ones.
#
# logfilename          [owner:group]    mode count size when  flags [/pid_file]
[sig_num]
/var/log/all.log                        600  7     *    @T00  J
/var/log/auth.log                       600  7     1000 @0101T JC
/var/log/console.log                    600  5     1000 *     J
/var/log/cron                           600  3     1000 *     JC
/var/log/daily.log                      640  7     *    @T00  JN
/var/log/debug.log                      600  7     1000 *     JC
/var/log/init.log                       644  3     1000 *     J
/var/log/kerberos.log                   600  7     1000 *     J
/var/log/maillog                        640  7     *    @T00  JC
/var/log/messages                       644  5     1000 @0101T JC
```

```
/var/log/monthly.log                    640  12    *    $M1D0 JN
/var/log/devd.log                       644  3     1000 *     JC
/var/log/security                       600  10    1000 *     JC
/var/log/utx.log                        644  3     *    @01T05 B
/var/log/weekly.log                     640  5     *    $W6D0 JN
/var/log/daemon.log                     644  5     1000 @0101T JC

<include> /etc/newsyslog.conf.d/[!.]*.conf
<include> /usr/local/etc/newsyslog.conf.d/[!.]*.conf
```

1. `logfilename` - Name of the system log file to be archived.
2. `[owner:group]` - This optional field specifies the owner and group for the archive file.
3. `mode` - Specify the file mode of the log file and archives. Valid mode bits are 0666. (That is, read and write permissions for the rotated log may be specified for the owner, group, and others.)
4. `count` - Specify the maximum number of archive files which may exist.
5. `size` - When the size of the log file reaches size in kilobytes, the log file will be trimmed as described above. If this field contains an asterisk ('*'), the log file will not be trimmed based on size.
6. `when` - Consist of an interval, a specific time, or both. Supported options in newsyslog.conf(5).
7. `flags` - Indicates the flags that newsyslog accepts, supported options in newsyslog.conf(5).
8. `[/pid_file]` - This optional field specifies the file name containing a daemon's process ID or to find a group process ID.
9. `[sig_num]` - This optional field specifies the signal that will be sent to the daemon process.



The last two fields are optional and specify the name of the Process ID (PID) file of a process and a signal number to send to that process when the file is rotated.

14.5.6. Configuring Remote Logging

Monitoring the log files of multiple hosts can become unwieldy as the number of systems increases. Configuring centralized logging can reduce some of the administrative burden of log file administration.

In FreeBSD, centralized log file aggregation, merging, and rotation can be configured using syslogd and newsyslog.

This section demonstrates an example configuration, where host `A`, named `logserv.example.com`, will collect logging information for the local network.

Host `B`, named `logclient.example.com`, will be configured to pass logging information to the logging server.

14.5.6.1. Log Server Configuration

A log server is a system that has been configured to accept logging information from other hosts.

Before configuring a log server, check the following:

- If there is a firewall between the logging server and any logging clients, ensure that the firewall ruleset allows UDP port 514 for both the clients and the server.
- The logging server and all client machines must have forward and reverse entries in the local DNS. If the network does not have a DNS server, create entries in each system's /etc/hosts. Proper name resolution is required so that log entries are not rejected by the logging server.

On the log server, edit /etc/syslog.conf to specify the name of the client to receive log entries from, the logging facility to be used, and the name of the log to store the host's log entries. This example adds the hostname of `B`, logs all facilities, and stores the log entries in /var/log/logclient.log.

Example 22. Sample Log Server Configuration

```
+logclient.example.com
*.*     /var/log/logclient.log
```

When adding multiple log clients, add a similar two-line entry for each client. More information about the available facilities may be found in syslog.conf(5).

Next, execute the following commands:

```
# sysrc syslogd_enable="YES"
# sysrc syslogd_flags="-a logclient.example.com -v -v"
```

The first entry starts syslogd at system boot. The second entry allows log entries from the specified client. The `-v -v` increases the verbosity of logged messages. This is useful for tweaking facilities as administrators are able to see what type of messages are being logged under each facility.

Multiple `-a` options may be specified to allow logging from multiple clients. IP addresses and whole netblocks may also be specified. Refer to syslogd(8) for a full list of possible options.

Finally, create the log file:

```
# touch /var/log/logclient.log
```

At this point, syslogd should be restarted and verified:

```
# service syslogd restart
# pgrep syslog
```

If a PID is returned, the server restarted successfully, and client configuration can begin. If the server did not restart, consult /var/log/messages for the error.

14.5.6.2. Log Client Configuration

A logging client sends log entries to a logging server on the network. The client also keeps a local copy of its own logs.

Once a logging server has been configured, execute the following commands on the logging client:

```
# sysrc syslogd_enable="YES"
# sysrc syslogd_flags="-s -v -v"
```

The first entry enables syslogd on boot up. The second entry prevents logs from being accepted by this client from other hosts (`-s`) and increases the verbosity of logged messages.

Next, define the logging server in the client's /etc/syslog.conf. In this example, all logged facilities are sent to a remote system, denoted by the `@` symbol, with the specified hostname:

```
*.*  @logserv.example.com
```

After saving the edit, restart syslogd for the changes to take effect:

```
# service syslogd restart
```

To test that log messages are being sent across the network, use logger(1) on the client to send a message to syslogd:

```
# logger "Test message from logclient"
```

This message should now exist both in /var/log/messages on the client and /var/log/logclient.log on the log server.

14.5.6.3. Debugging Log Servers

If no messages are being received on the log server, the cause is most likely a network connectivity issue, a hostname resolution issue, or a typo in a configuration file. To isolate the cause, ensure that both the logging server and the logging client are able to `ping` each other using the hostname specified in their /etc/rc.conf. If this fails, check the network cabling, the firewall ruleset, and the hostname entries in the DNS server or /etc/hosts on both the logging server and clients. Repeat until the `ping` is successful from both hosts.

If the `ping` succeeds on both hosts but log messages are still not being received, temporarily increase logging verbosity to narrow down the configuration issue. In the following example, /var/log/logclient.log on the logging server is empty and /var/log/messages on the logging client does not indicate a reason for the failure.

To increase debugging output, edit the `syslogd_flags` entry on the logging server and issue a restart:

```
sysrc syslogd_flags="-d -a logclient.example.com -v -v"
```

```
# service syslogd restart
```

Debugging data similar to the following will flash on the console immediately after the restart:

```
logmsg: pri 56, flags 4, from logserv.example.com, msg syslogd: restart
syslogd: restarted
logmsg: pri 6, flags 4, from logserv.example.com, msg syslogd: kernel boot file is
/boot/kernel/kernel
Logging to FILE /var/log/messages
syslogd: kernel boot file is /boot/kernel/kernel
cvthname(192.168.1.10)
validate: dgram from IP 192.168.1.10, port 514, name logclient.example.com;
rejected in rule 0 due to name mismatch.
```

In this example, the log messages are being rejected due to a typo which results in a hostname mismatch. The client's hostname should be `logclient`, not `logclien`. Fix the typo, issue a restart, and verify the results:

```
# service syslogd restart
```

The output should be similar to the following:

```
logmsg: pri 56, flags 4, from logserv.example.com, msg syslogd: restart
syslogd: restarted
logmsg: pri 6, flags 4, from logserv.example.com, msg syslogd: kernel boot file is
/boot/kernel/kernel
syslogd: kernel boot file is /boot/kernel/kernel
logmsg: pri 166, flags 17, from logserv.example.com,
msg Dec 10 20:55:02 <syslog.err> logserv.example.com syslogd: exiting on signal 2
cvthname(192.168.1.10)
validate: dgram from IP 192.168.1.10, port 514, name logclient.example.com;
accepted in rule 0.
logmsg: pri 15, flags 0, from logclient.example.com, msg Dec 11 02:01:28 trhodes: Test
message 2
Logging to FILE /var/log/logclient.log
Logging to FILE /var/log/messages
```

At this point, the messages are being properly received and placed in the correct file.

14.5.6.4. Security Considerations

As with any network service, security requirements should be considered before implementing a logging server. Log files may contain sensitive data about services enabled on the local host, user

accounts, and configuration data. Network data sent from the client to the server will not be encrypted or password protected. If a need for encryption exists, consider using security/stunnel, which will transmit the logging data over an encrypted tunnel.

Local security is also an issue. Log files are not encrypted during use or after log rotation. Local users may access log files to gain additional insight into system configuration. Setting proper permissions on log files is critical. The built-in log rotator, newsyslog, supports setting permissions on newly created and rotated log files. Setting log files to mode `600` should prevent unwanted access by local users. Refer to newsyslog.conf(5) for additional information.

14.6. Power and Resource Management

It is important to utilize hardware resources in an efficient manner. Power and resource management allows the operating system to monitor system limits and to possibly run some actions triggered by events related to those limits.

14.6.1. ACPI configuration

On FreeBSD the management of these resources is managed by the acpi(4) kernel device.



In FreeBSD the acpi(4) driver is loaded by default at system boot.

This driver **cannot be unloaded after boot** because the system bus uses it for various hardware interactions.

In addition to acpi(4), FreeBSD has several dedicated kernel modules for various ACPI vendor subsystems. These modules will add some extra functionality like fan speed, keyboard backlit or screen brightness.

The list can be obtained by running the following command:

```
% ls /boot/kernel | grep acpi
```

The output should be similar to the following:

```
acpi_asus.ko
acpi_asus_wmi.ko
acpi_dock.ko
acpi_fujitsu.ko
acpi_hp.ko
acpi_ibm.ko
acpi_panasonic.ko
acpi_sony.ko
acpi_toshiba.ko
acpi_video.ko
acpi_wmi.ko
sdhci_acpi.ko
```

```
uacpi.ko
```

In the event that, for example, an IBM/Lenovo laptop is used, it will be necessary to load the module acpi_ibm(4) by executing the following command:

```
# kldload acpi_ibm
```

And add this line to /boot/loader.conf to load it at boot:

```
acpi_ibm_load="YES"
```

An alternative to the acpi_video(4) module is the backlight(9) driver. It provides a generic way for handling a panel backlight. The default GENERIC kernel includes this driver. The backlight(8) utility can be used to query and adjust the brightness of the panel backlight. In this example the brightness is decreased by 10%:

```
% backlight decr 10
```

14.6.2. CPU Power Management

CPU is the most consuming part of the system. Knowing how to improve CPU efficiency is a fundamental part of our system in order to save energy.

In order to make proper use of the machine's resources in a correct way, FreeBSD supports technologies such as Intel Turbo Boost, AMD Turbo Core, Intel Speed Shift among others through the use of powerd(8) and cpufreq(4).

The first step will be to obtain the CPU information by executing the following command:

```
% sysctl dev.cpu.0 ①
```

① In this case the `0` digit represents the first core of the CPU.

The output should be similar to the following:

```
dev.cpu.0.cx_method: C1/mwait/hwc C2/mwait/hwc C3/mwait/hwc/bma
dev.cpu.0.cx_usage_counters: 3507294 0 0
dev.cpu.0.cx_usage: 100.00% 0.00% 0.00% last 3804us
dev.cpu.0.cx_lowest: C3 ①
dev.cpu.0.cx_supported: C1/1/1 C2/2/1 C3/3/57 ②
dev.cpu.0.freq_levels: 2267/35000 2266/35000 1600/15000 800/12000 ③
dev.cpu.0.freq: 1600 ④
dev.cpu.0.temperature: 40.0C ⑤
dev.cpu.0.coretemp.throttle_log: 0
dev.cpu.0.coretemp.tjmax: 105.0C
```

```
dev.cpu.0.coretemp.resolution: 1
dev.cpu.0.coretemp.delta: 65
dev.cpu.0.%parent: acpi0
dev.cpu.0.%pnpinfo: _HID=none _UID=0 _CID=none
dev.cpu.0.%location: handle=\_PR_.CPU0
dev.cpu.0.%driver: cpu
dev.cpu.0.%desc: ACPI CPU
```

① Lowest Cx state to use for idling the CPU.

② CPU supported Cx states.

③ Currently available levels for the CPU (frequency/power usage).

④ Current active CPU frequency in MHz.

⑤ Current temperature of the CPU.



If the temperature information is not displayed, load the coretemp(4) module. In case of using an AMD CPU, load the amdtemp(4) module.

Once the CPU information is available the easiest way to configure power saving is to let powerd(8) take over.

Enable powerd(8) service in /etc/rc.conf to start at system boot:

```
# sysrc powerd_enable=YES
```

It will also be necessary to indicate certain parameters to powerd(8) to tell it how to manage the state of the CPU executing the following command:

```
# sysrc powerd_flags="-a hiadaptive -i 25 -r 85 -N"
```

1. `-a`: Selects the mode to use while on AC power.
2. `hiadaptive`: Operation mode. More info at powerd(8).
3. `-i`: Specifies the CPU load percent level when adaptive mode should begin to degrade performance to save power.
4. `-r`: Specifies the CPU load percent level where adaptive mode should consider the CPU running and increase performance.
5. `-N`: Treat "nice" time as idle for the purpose of load calculation; i.e., do not increase the CPU frequency if the CPU is only busy with "nice" processes.

And then enable the service executing the following command:

```
# service powerd start
```

14.6.3. CPU Frequency Control

FreeBSD includes a generic cpufreq(4) driver to allow the administrator, or software such as powerd(8) and sysutils/powerdxx, to manage the frequency of the CPU to achieve the desired balance between performance and economy. A lower setting will save power while reducing the heat generated by the CPU. A higher setting will increase performance at the cost of using additional power and generating more heat.

14.6.4. Intel® Enhanced Speed Step™

The Intel® Enhanced Speed Step™ driver, est(4), replaces the generic cpufreq(4) driver for CPUs that provide this feature. The CPU frequency can be statically adjusted using sysctl(8), or with the `/etc/rc.d/power_profile` startup script. Additional software, such as powerd(8) or sysutils/powerdxx, can be used to automatically adjust the CPU frequency based on processor utilization.

Each supported frequency, along with its expected power consumption, can be listed by examining the sysctl(3) tree:

```
# sysctl dev.cpufreq.0.freq_driver dev.cpu.0.freq_levels dev.cpu.0.freq
```

The output should be similar to the following:

```
dev.cpufreq.0.freq_driver: est0
dev.cpu.0.freq_levels: 3001/53000 3000/53000 2900/50301 2700/46082 2600/43525
2400/39557 2300/37137 2100/33398 2000/31112 1800/27610 1700/25455 1500/22171
1400/20144 1200/17084 1100/15181 900/12329 800/10550
dev.cpu.0.freq: 800
```

A frequency 1 MHz higher than the maximum frequency of the CPU indicates the Intel® Turbo Boost™ feature.

14.6.5. Intel Speed Shift™

Users running newer Intel® CPUs may find some differences in dynamic frequency control when upgrading to FreeBSD 13. A new driver for the Intel® Speed Shift™ feature set, available on certain SKUs, exposes the ability for the hardware to dynamically vary the core frequencies, including on a per core basis. FreeBSD 13 comes with the hwpstate_intel(4) driver to automatically enable Speed Shift™ control on equipped CPUs, replacing the older Enhanced Speed Step™ est(4) driver. The sysctl(8) `dev.cpufreq.%d.freq_driver` will indicate if the system is using Speed Shift.

To determine which frequency control driver is being used, examining the `dev.cpufreq.0.freq_driver` oid.

```
# sysctl dev.cpufreq.0.freq_driver
```

The output should be similar to the following:

```
dev.cpufreq.0.freq_driver: hwpstate_intel0
```

This indicates that the new hwpstate_intel(4) driver is in use. On such systems, the oid `dev.cpu.%d.freq_levels` will show only the maximum CPU frequency, and will indicate a power consumption level of `-1`.

The current CPU frequency can be determined by examining the `dev.cpu.%d.freq` oid.

```
# sysctl dev.cpu.0.freq_levels dev.cpu.0.freq
```

The output should be similar to the following:

```
dev.cpu.0.freq_levels: 3696/-1
dev.cpu.0.freq: 898
```

For more information, including on how to balance performance and energy use, and on how to disable this driver, refer to the man page hwpstate_intel(4).



Users accustomed to using powerd(8) or sysutils/powerdxx will find these utilities have been superseded by the hwpstate_intel(4) driver and no longer work as expected.

14.6.6. Graphics Card Power Management

Graphics cards have become a fundamental part of computing in recent years. Some graphics cards may have excessive power consumption. FreeBSD allows certain configurations to improve power consumption.

In case of using a Intel® graphics card with the graphics/drm-kmod driver these options can be added to /boot/loader.conf:

```
compat.linuxkpi.fastboot=1 ①
compat.linuxkpi.enable_dc=2 ②
compat.linuxkpi.enable_fbc=1 ③
```

① Try to skip unnecessary mode sets at boot time.

② Enable power-saving display C-states.

③ Enable frame buffer compression for power savings

14.6.7. Suspend/Resume

The suspend/resume function allows the machine to be kept in a state in which there is no a big

energy consumption and allows the system to be resumed without having to lose the state of the running programs.



In order for the suspend/resume functionality to work correctly the graphics drivers must be loaded on the system. In non-KMS-supported graphics cards sc(4) must be used not to break the suspend/resume functionality.

More information about which driver to use and how to configure it can be found at the The X Window System chapter.

acpi(4) supports the next list of sleep states:

Table 27. Supported Sleep States

S1	**Quick suspend to RAM. The CPU enters a lower power state, but most peripherals are left running.**
S2	Lower power state than S1, but with the same basic characteristics. Not supported by many systems.
S3 (Sleep mode)	Suspend to RAM. Most devices are powered off, and the system stops running except for memory refresh.
S4 (Hibernation)	Suspend to disk. All devices are powered off, and the system stops running. When resuming, the system starts as if from a cold power on. **Not yet supported by FreeBSD**.
S5	System shuts down cleanly and powers off.

14.6.7.1. Configuring Suspend/Resume

The first step will be to know which type of sleep states supports the hardware we are using executing the following command:

```
% sysctl hw.acpi.supported_sleep_state
```

The output should be similar to the following:

```
hw.acpi.supported_sleep_state: S3 S4 S5
```



As stated above FreeBSD does **not** yet support the S4 state.

acpiconf(8) can be used to check if the S3 state works correctly by running the following command, if it succeeds, the screen should go black and the machine will turn off:

```
# acpiconf -s 3
```

In the vast majority of cases the Suspend/Resume functionality wants to be used on a laptop.

FreeBSD can be configured to enter the S3 state when closing the lid by adding the following line to the /etc/sysctl.conf file.

```
hw.acpi.lid_switch_state=S3
```

14.6.7.2. Troubleshooting in Suspend/Resume

A lot of effort has been made to make the Suspend and Resume functions work properly and in the best way on FreeBSD. But currently the Suspend and Resume functions only work properly on some specific laptops.

Some checks can be done in case it doesn't work properly.

In some cases it is enough to turn off the bluetooth. In others it is enough loading the correct driver for the graphics card, etc.

In case it doesn't work correctly, some tips can be found on the FreeBSD Wiki in the section Suspend/Resume.

14.7. Adding Swap Space

Sometimes a FreeBSD system requires more swap space. This section describes two methods to increase swap space: adding swap to an existing partition or new hard drive, and creating a swap file on an existing file system.

For information on how to encrypt swap space, which options exist, and why it should be done, refer to "Encrypting Swap".

14.7.1. Swap on a New Hard Drive or Existing Partition

Adding a new drive for swap gives better performance than using a partition on an existing drive. Setting up partitions and drives is explained in Adding Disks while Designing the Partition Layout discusses partition layouts and swap partition size considerations.



It is possible to use any partition not currently mounted, even if it already contains data. Using `swapon` on a partition that contains data will overwrite and destroy that data. Make sure that the partition to be added as swap is really the intended partition before running `swapon`.

swapon(8) can be used to add a swap partition to the system executing the following command:

```
# swapon /dev/ada1p2
```

To automatically add this swap partition on boot, add an entry to /etc/fstab:

```
/dev/ada1p2 none swap sw 0 0
```

See fstab(5) for an explanation of the entries in /etc/fstab.

14.7.2. Creating a Swap File

These examples create a 512M swap file called /usr/swap0.



Swap files on ZFS file systems are strongly discouraged, as swapping can lead to system hangs.

The first step is to create the swap file:

```
# dd if=/dev/zero of=/usr/swap0 bs=1m count=512
```

The second step is to put the proper permissions on the new file:

```
# chmod 0600 /usr/swap0
```

The third step is to inform the system about the swap file by adding a line to /etc/fstab:

```
md none swap sw,file=/usr/swap0,late 0 0
```

Swap space will be added on system startup. To add swap space immediately, use swapon(8):

```
# swapon -aL
```

Chapter 15. The FreeBSD Booting Process

15.1. Synopsis

The process of starting a computer and loading the operating system is referred to as "the bootstrap process", or "booting". FreeBSD's boot process provides a great deal of flexibility in customizing what happens when the system starts, including the ability to select from different operating systems installed on the same computer, different versions of the same operating system, or a different installed kernel.

This chapter details the configuration options that can be set. It demonstrates how to customize the FreeBSD boot process, including everything that happens until the FreeBSD kernel has started, probed for devices, and started init(8). This occurs when the text color of the boot messages changes from bright white to grey.

Read this chapter to learn:

- The components of the FreeBSD bootstrap system and how they interact.
- The options that can be passed to the components in the FreeBSD bootstrap in order to control the boot process.
- The basics of setting device hints.
- How to boot into single- and multi-user mode and how to properly shut down a FreeBSD system.



This chapter only describes the boot process for FreeBSD running on x86 and amd64 systems.

15.2. FreeBSD Boot Process

Turning on a computer and starting the operating system poses an interesting dilemma. By definition, the computer does not know how to do anything until the operating system is started. This includes running programs from the disk. If the computer can not run a program from the disk without the operating system, and the operating system programs are on the disk, how is the operating system started?

This problem parallels one in the book The Adventures of Baron Munchausen. A character had fallen part way down a manhole, and pulled himself out by grabbing his bootstraps and lifting. In the early days of computing, the term *bootstrap* was applied to the mechanism used to load the operating system. It has since become shortened to "booting".

On x86 hardware, the Basic Input/Output System (BIOS) is responsible for loading the operating system. The BIOS looks on the hard disk for the Master Boot Record (MBR), which must be located in a specific place on the disk. The BIOS has enough knowledge to load and run the MBR, and assumes that the MBR can then carry out the rest of the tasks involved in loading the operating system, possibly with the help of the BIOS.



FreeBSD provides for booting from both the older MBR standard, and the newer

GUID Partition Table (GPT). GPT partitioning is often found on computers with the Unified Extensible Firmware Interface (UEFI). However, FreeBSD can boot from GPT partitions even on machines with only a legacy BIOS with gptboot(8). Work is under way to provide direct UEFI booting.

The code within the MBR is typically referred to as a *boot manager*, especially when it interacts with the user. The boot manager usually has more code in the first track of the disk or within the file system. Examples of boot managers include the standard FreeBSD boot manager boot0, also called Boot Easy, and GNU GRUB, which is used by many Linux® distributions.



Users of GRUB should refer to GNU-provided documentation.

If only one operating system is installed, the MBR searches for the first bootable (active) slice on the disk, and then runs the code on that slice to load the remainder of the operating system. When multiple operating systems are present, a different boot manager can be installed to display a list of operating systems so the user can select one to boot.

The remainder of the FreeBSD bootstrap system is divided into three stages. The first stage knows just enough to get the computer into a specific state and run the second stage. The second stage can do a little bit more, before running the third stage. The third stage finishes the task of loading the operating system. The work is split into three stages because the MBR puts limits on the size of the programs that can be run at stages one and two. Chaining the tasks together allows FreeBSD to provide a more flexible loader.

The kernel is then started and begins to probe for devices and initialize them for use. Once the kernel boot process is finished, the kernel passes control to the user process init(8), which makes sure the disks are in a usable state, starts the user-level resource configuration which mounts file systems, sets up network cards to communicate on the network, and starts the processes which have been configured to run at startup.

This section describes these stages in more detail and demonstrates how to interact with the FreeBSD boot process.

15.2.1. The Boot Manager

The boot manager code in the MBR is sometimes referred to as *stage zero* of the boot process. By default, FreeBSD uses the boot0 boot manager.

The MBR installed by the FreeBSD installer is based on /boot/boot0. The size and capability of boot0 is restricted to 446 bytes due to the slice table and `0x55AA` identifier at the end of the MBR. If boot0 and multiple operating systems are installed, a message similar to this example will be displayed at boot time:

Example 23. boot0 Screenshot

```
F1 Win
F2 FreeBSD
```

```
Default: F2
```

Other operating systems will overwrite an existing MBR if they are installed after FreeBSD. If this happens, or to replace the existing MBR with the FreeBSD MBR, use the following command:

```
# fdisk -B -b /boot/boot0 device
```

where *device* is the boot disk, such as ad0 for the first IDE disk, ad2 for the first IDE disk on a second IDE controller, or da0 for the first SCSI disk. To create a custom configuration of the MBR, refer to boot0cfg(8).

15.2.2. Stage One and Stage Two

Conceptually, the first and second stages are part of the same program on the same area of the disk. Due to space constraints, they have been split into two, but are always installed together. They are copied from the combined /boot/boot by the FreeBSD installer or `bsdlabel`.

These two stages are located outside file systems, in the first track of the boot slice, starting with the first sector. This is where boot0, or any other boot manager, expects to find a program to run which will continue the boot process.

The first stage, boot1, is very simple, since it can only be 512 bytes in size. It knows just enough about the FreeBSD *bsdlabel*, which stores information about the slice, to find and execute boot2.

Stage two, boot2, is slightly more sophisticated, and understands the FreeBSD file system enough to find files. It can provide a simple interface to choose the kernel or loader to run. It runs loader, which is much more sophisticated and provides a boot configuration file. If the boot process is interrupted at stage two, the following interactive screen is displayed:

Example 24. boot2 Screenshot

```
>> FreeBSD/i386 BOOT
Default: 0:ad(0,a)/boot/loader
boot:
```

To replace the installed boot1 and boot2, use `bsdlabel`, where *diskslice* is the disk and slice to boot from, such as ad0s1 for the first slice on the first IDE disk:

```
# bsdlabel -B diskslice
```



If just the disk name is used, such as ad0, `bsdlabel` will create the disk in "dangerously dedicated mode", without slices. This is probably not the desired action, so double check the *diskslice* before pressing `Return`.

15.2.3. Stage Three

The loader is the final stage of the three-stage bootstrap process. It is located on the file system, usually as /boot/loader.

The loader is intended as an interactive method for configuration, using a built-in command set, backed up by a more powerful interpreter which has a more complex command set.

During initialization, loader will probe for a console and for disks, and figure out which disk it is booting from. It will set variables accordingly, and an interpreter is started where user commands can be passed from a script or interactively.

The loader will then read /boot/loader.rc, which by default reads in /boot/defaults/loader.conf which sets reasonable defaults for variables and reads /boot/loader.conf for local changes to those variables. loader.rc then acts on these variables, loading whichever modules and kernel are selected.

Finally, by default, loader issues a 10 second wait for key presses, and boots the kernel if it is not interrupted. If interrupted, the user is presented with a prompt which understands the command set, where the user may adjust variables, unload all modules, load modules, and then finally boot or reboot. Loader Built-In Commands lists the most commonly used loader commands. For a complete discussion of all available commands, refer to loader(8).

Table 28. Loader Built-In Commands

Variable	Description
autoboot *seconds*	Proceeds to boot the kernel if not interrupted within the time span given, in seconds. It displays a countdown, and the default time span is 10 seconds.
boot [`-options`] [`kernelname`]	Immediately proceeds to boot the kernel, with any specified options or kernel name. Providing a kernel name on the command-line is only applicable after an `unload` has been issued. Otherwise, the previously-loaded kernel will be used. If *kernelname* is not provided with the full path, it will be searched under */boot/kernel* and */boot/modules*.
boot-conf	Goes through the same automatic configuration of modules based on specified variables, most commonly `kernel`. This only makes sense if `unload` is used first, before changing some variables.
help [`topic`]	Shows help messages read from /boot/loader.help. If the topic given is `index`, the list of available topics is displayed.
include `filename` ...	Reads the specified file and interprets it line by line. An error immediately stops the `include`.
load [-t `type`] `filename`	Loads the kernel, kernel module, or file of the type given, with the specified filename. Any arguments after *filename* are passed to the file. If *filename* is not provided with the full path, it will be searched under */boot/kernel* and */boot/modules*.
ls [-l] [`path`]	Displays a listing of files in the given path, or the root directory, if the path is not specified. If `-l` is specified, file sizes will also be shown.

Variable	Description
lsdev [-v]	Lists all of the devices from which it may be possible to load modules. If -v is specified, more details are printed.
lsmod [-v]	Displays loaded modules. If -v is specified, more details are shown.
more filename	Displays the files specified, with a pause at each LINES displayed.
reboot	Immediately reboots the system.
set variable, set variable=value	Sets the specified environment variables.
unload	Removes all loaded modules.

Here are some practical examples of loader usage. To boot the usual kernel in single-user mode:

```
boot -s
```

To unload the usual kernel and modules and then load the previous or another, specified kernel:

```
unload
load /path/to/kernelfile
```

Use the full path /boot/GENERIC/kernel to refer to the default kernel that comes with an installation, or /boot/kernel.old/kernel, to refer to the previously installed kernel before a system upgrade or before configuring a custom kernel.

Use the following to load the usual modules with another kernel. Note that in this case it is not necessary to provide the full path, only the file name:

```
unload
set kernel="mykernel"
boot-conf
```

To load an automated kernel configuration script:

```
load -t userconfig_script /boot/kernel.conf
```

15.2.4. Last Stage

Once the kernel is loaded by either loader or by boot2, which bypasses loader, it examines any boot flags and adjusts its behavior as necessary. Kernel Interaction During Boot lists the commonly used boot flags. Refer to boot(8) for more information on the other boot flags.

Table 29. Kernel Interaction During Boot

Option	Description
`-a`	During kernel initialization, ask for the device to mount as the root file system.
`-C`	Boot the root file system from a CDROM.
`-s`	Boot into single-user mode.
`-v`	Be more verbose during kernel startup.

Once the kernel has finished booting, it passes control to the user process init(8), which is located at /sbin/init, or the program path specified in the `init_path` variable in `loader`. This is the last stage of the boot process.

The boot sequence makes sure that the file systems available on the system are consistent. If a UFS file system is not, and `fsck` cannot fix the inconsistencies, init drops the system into single-user mode so that the system administrator can resolve the problem directly. Otherwise, the system boots into multi-user mode.

15.2.4.1. Single-User Mode

A user can specify this mode by booting with `-s` or by setting the `boot_single` variable in loader. It can also be reached by running `shutdown now` from multi-user mode. Single-user mode begins with this message:

```
Enter full pathname of shell or RETURN for /bin/sh:
```

If the user presses `Enter`, the system will enter the default POSIX® shell. To specify a different shell, input the full path to the shell.

Single-user mode is usually used to repair a system that will not boot due to an inconsistent file system or an error in a boot configuration file. It can also be used to reset the `root` password when it is unknown. These actions are possible as the single-user mode prompt gives full, local access to the system and its configuration files. There is no networking in this mode.

While single-user mode is useful for repairing a system, it poses a security risk unless the system is in a physically secure location. By default, any user who can gain physical access to a system will have full control of that system after booting into single-user mode.

If the system `console` is changed to `insecure` in /etc/ttys, the system will first prompt for the `root` password before initiating single-user mode. This adds a measure of security while removing the ability to reset the `root` password when it is unknown.

Example 25. Configuring an Insecure Console in /etc/ttys

```
# name  getty                           type    status          comments
#
# If console is marked "insecure", then init will ask for the root password
# when going to single-user mode.
```

```
console none                            unknown off insecure
```

An `insecure` console means that physical security to the console is considered to be insecure, so only someone who knows the `root` password may use single-user mode.

15.2.4.2. Multi-User Mode

If init finds the file systems to be in order, or once the user has finished their commands in single-user mode and has typed `exit` to leave single-user mode, the system enters multi-user mode, in which it starts the resource configuration of the system.

The resource configuration system reads in configuration defaults from /etc/defaults/rc.conf and system-specific details from /etc/rc.conf. It then proceeds to mount the system file systems listed in /etc/fstab. It starts up networking services, miscellaneous system daemons, then the startup scripts of locally installed packages.

To learn more about the resource configuration system, refer to rc(8) and examine the scripts located in /etc/rc.d.

15.3. Device Hints

During initial system startup, the boot loader(8) reads device.hints(5). This file stores kernel boot information known as variables, sometimes referred to as "device hints". These "device hints" are used by device drivers for device configuration.

Device hints may also be specified at the Stage 3 boot loader prompt, as demonstrated in Stage Three. Variables can be added using `set`, removed with `unset`, and viewed `show`. Variables set in /boot/device.hints can also be overridden. Device hints entered at the boot loader are not permanent and will not be applied on the next reboot.

Once the system is booted, kenv(1) can be used to dump all of the variables.

The syntax for /boot/device.hints is one variable per line, using the hash "#" as comment markers. Lines are constructed as follows:

```
hint.driver.unit.keyword="value"
```

The syntax for the Stage 3 boot loader is:

```
set hint.driver.unit.keyword=value
```

where `driver` is the device driver name, `unit` is the device driver unit number, and `keyword` is the hint keyword. The keyword may consist of the following options:

- `at`: specifies the bus which the device is attached to.
- `port`: specifies the start address of the I/O to be used.

- `irq`: specifies the interrupt request number to be used.
- `drq`: specifies the DMA channel number.
- `maddr`: specifies the physical memory address occupied by the device.
- `flags`: sets various flag bits for the device.
- `disabled`: if set to `1` the device is disabled.

Since device drivers may accept or require more hints not listed here, viewing a driver's manual page is recommended. For more information, refer to device.hints(5), kenv(1), loader.conf(5), and loader(8).

15.4. Shutdown Sequence

Upon controlled shutdown using shutdown(8), init(8) will attempt to run the script /etc/rc.shutdown, and then proceed to send all processes the `TERM` signal, and subsequently the `KILL` signal to any that do not terminate in a timely manner.

To power down a FreeBSD machine on architectures and systems that support power management, use `shutdown -p now` to turn the power off immediately. To reboot a FreeBSD system, use `shutdown -r now`. One must be `root` or a member of `operator` in order to run shutdown(8). One can also use halt(8) and reboot(8). Refer to their manual pages and to shutdown(8) for more information.

Modify group membership by referring to "Users and Basic Account Management".



Power management requires acpi(4) to be loaded as a module or statically compiled into a custom kernel.

Chapter 16. Security

16.1. Synopsis

Hundreds of standard practices have been authored about how to secure systems and networks, and as a user of FreeBSD, understanding how to protect against attacks and intruders is a must.

In this chapter, several fundamentals and techniques will be discussed. The FreeBSD system comes with multiple layers of security, and many more third party utilities may be added to enhance security.

This chapter covers:

- Basic FreeBSD system security concepts.
- The various crypt mechanisms available in FreeBSD.
- How to configure TCP Wrappers for use with inetd(8).
- How to set up Kerberos on FreeBSD.
- How to configure and use OpenSSH on FreeBSD.
- How to use OpenSSL on FreeBSD.
- How to use file system ACLs.
- How to use pkg to audit third party software packages installed from the Ports Collection.
- How to utilize FreeBSD security advisories.
- What Process Accounting is and how to enable it on FreeBSD.
- How to control user resources using login classes or the resource limits database.
- What is Capsicum and a basic example.

Certain topics due to their complexity are found in dedicated chapters such as Firewalls, Mandatory Access Control and articles like VPN over IPsec.

16.2. Introduction

Security is everyone's responsibility. A weak entry point in any system could allow intruders to gain access to critical information and cause havoc on an entire network. One of the core principles of information security is the CIA triad, which stands for the Confidentiality, Integrity, and Availability of information systems.

The CIA triad is a bedrock concept of computer security as customers and users expect their data to be protected. For example, a customer expects that their credit card information is securely stored (confidentiality), that their orders are not changed behind the scenes (integrity), and that they have access to their order information at all times (availability).

To provide CIA, security professionals apply a defense in depth strategy. The idea of defense in depth is to add several layers of security to prevent one single layer failing and the entire security

system collapsing. For example, a system administrator cannot simply turn on a firewall and consider the network or system secure. One must also audit accounts, check the integrity of binaries, and ensure malicious tools are not installed. To implement an effective security strategy, one must understand threats and how to defend against them.

What is a threat as it pertains to computer security? Threats are not limited to remote attackers who attempt to access a system without permission from a remote location. Threats also include employees, malicious software, unauthorized network devices, natural disasters, security vulnerabilities, and even competing corporations.

Systems and networks can be accessed without permission, sometimes by accident, or by remote attackers, and in some cases, via corporate espionage or former employees. As a user, it is important to prepare for and admit when a mistake has led to a security breach and report possible issues to the security team. As an administrator, it is important to know of the threats and be prepared to mitigate them.

When applying security to systems, it is recommended to start by securing the basic accounts and system configuration, and then to secure the network layer so that it adheres to the system policy and the organization's security procedures. Many organizations already have a security policy that covers the configuration of technology devices. The policy should include the security configuration of workstations, desktops, mobile devices, phones, production servers, and development servers. In many cases, standard operating procedures (SOPs) already exist. When in doubt, ask the security team.

16.3. Securing Accounts

Maintaining secure accounts in FreeBSD is crucial for data confidentiality, system integrity, and privilege separation, as it prevents unauthorized access, malware, and data breaches while ensuring compliance and protecting an organization's reputation.

16.3.1. Preventing Logins

In securing a system, a good starting point is an audit of accounts. Disable any accounts that do not need login access.



Ensure that `root` has a strong password and that this password is not shared.

To deny login access to accounts, two methods exist.

The first is to lock the account, this example shows how to lock the `imani` account:

```
# pw lock imani
```

The second method is to prevent login access by changing the shell to /usr/sbin/nologin. The nologin(8) shell prevents the system from assigning a shell to the user when they attempt to login.

Only the superuser can change the shell for other users:

```
# chsh -s /usr/sbin/nologin imani
```

16.3.2. Password Hashes

Passwords are a necessary evil of technology. When they must be used, they should be complex and a powerful hash mechanism should be used to encrypt the version that is stored in the password database. FreeBSD supports several algorithms, including SHA256, SHA512 and Blowfish hash algorithms in its `crypt()` library, see crypt(3) for details.

The default of SHA512 should not be changed to a less secure hashing algorithm, but can be changed to the more secure Blowfish algorithm.



Blowfish is not part of AES and is not considered compliant with any Federal Information Processing Standards (FIPS). Its use may not be permitted in some environments.

To determine which hash algorithm is used to encrypt a user's password, the superuser can view the hash for the user in the FreeBSD password database. Each hash starts with a symbol which indicates the type of hash mechanism used to encrypt the password.

If DES is used, there is no beginning symbol. For MD5, the symbol is `$`. For SHA256 and SHA512, the symbol is `$6$`. For Blowfish, the symbol is `$2a$`. In this example, the password for `imani` is hashed using the default SHA512 algorithm as the hash starts with `$6$`. Note that the encrypted hash, not the password itself, is stored in the password database:

```
# grep imani /etc/master.passwd
```

The output should be similar to the following:

```
imani:$6$pzIjSvCAn.PBYQBA$PXpSeWPx3g5kscj3IMiM7tUEUSPmGexxta.8Lt9TGSi2lNQqYGKszsBPuGME
0:1001:1001::0:0:imani:/usr/home/imani:/bin/sh
```

The hash mechanism is set in the user's login class.

The following command can be run to check which hash mechanism is currently being used:

```
% grep passwd_format /etc/login.conf
```

The output should be similar to the following:

```
:passwd_format=sha512:\
```

For example, to change the algorithm to Blowfish, modify that line to look like this:

```
:passwd_format=blf:\
```

Then, cap_mkdb(1) must be executed to upgrade the login.conf database:

```
# cap_mkdb /etc/login.conf
```

Note that this change will not affect any existing password hashes. This means that all passwords should be re-hashed by asking users to run `passwd` in order to change their password.

16.3.3. Password Policy Enforcement

Enforcing a strong password policy for local accounts is a fundamental aspect of system security. In FreeBSD, password length, password strength, and password complexity can be implemented using built-in Pluggable Authentication Modules (PAM).

This section demonstrates how to configure the minimum and maximum password length and the enforcement of mixed characters using the pam_passwdqc(8) module. This module is enforced when a user changes their password.

To configure this module, become the superuser and uncomment the line containing `pam_passwdqc.so` in /etc/pam.d/passwd.

Then, edit that line to match the password policy:

```
password        requisite       pam_passwdqc.so
min=disabled,disabled,disabled,12,10 similar=deny retry=3 enforce=users
```

The explanation of the parameters can be found in pam_passwdqc(8).

Once this file is saved, a user changing their password will see a message similar to the following:

```
% passwd
```

The output should be similar to the following:

```
Changing local password for user
Old Password:

You can now choose the new password.
A valid password should be a mix of upper and lower case letters,
digits and other characters.  You can use a 12 character long
password with characters from at least 3 of these 4 classes, or
a 10 character long password containing characters from all the
classes.  Characters that form a common pattern are discarded by
the check.
```

```
Alternatively, if no one else can see your terminal now, you can
pick this as your password: "trait-useful&knob".
Enter new password:
```

If a password that does not match the policy is entered, it will be rejected with a warning and the user will have an opportunity to try again, up to the configured number of retries.

If the organization's policy requires passwords to expire, FreeBSD supports the `passwordtime` in the user's login class in /etc/login.conf

The `default` login class contains an example:

```
#       :passwordtime=90d:\
```

So, to set an expiry of 90 days for this login class, remove the comment symbol (#), save the edit, and execute the following command:

```
# cap_mkdb /etc/login.conf
```

To set the expiration on individual users, pass an expiration date or the number of days to expiry and a username to `pw`:

```
# pw usermod -p 30-apr-2025 -n user
```

As seen here, an expiration date is set in the form of day, month, and year. For more information, see pw(8).

16.3.4. Shared Administration with sudo

System administrators often need the ability to grant enhanced permissions to users so they may perform privileged tasks. The idea that team members are provided access to a FreeBSD system to perform their specific tasks opens up unique challenges to every administrator. These team members only need a subset of access beyond normal end user levels; however, they almost always tell management they are unable to perform their tasks without superuser access. Thankfully, there is no reason to provide such access to end users because tools exist to manage this exact requirement.



Even administrators should limit their privileges when not needed.

Up to this point, the security chapter has covered permitting access to authorized users and attempting to prevent unauthorized access. Another problem arises once authorized users have access to the system resources. In many cases, some users may need access to application startup scripts, or a team of administrators need to maintain the system. Traditionally, the standard users and groups, file permissions, and even the su(1) command would manage this access. And as applications required more access, as more users needed to use system resources, a better solution

was required. The most used application is currently Sudo.

Sudo allows administrators to configure more rigid access to system commands and provide for some advanced logging features. As a tool, it is available from the Ports Collection as security/sudo or by use of the pkg(8) utility.

Execute the following command to install it:

```
# pkg install sudo
```

After the installation is complete, the installed `visudo` will open the configuration file with a text editor. Using `visudo` is highly recommended as it comes with a built in syntax checker to verify there are no errors before the file is saved.

The configuration file is made up of several small sections which allow for extensive configuration. In the following example, web application maintainer, user1, needs to start, stop, and restart the web application known as *webservice*. To grant this user permission to perform these tasks, add this line to the end of /usr/local/etc/sudoers:

```
user1   ALL=(ALL)       /usr/sbin/service webservice *
```

The user may now start *webservice* using this command:

```
% sudo /usr/sbin/service webservice start
```

While this configuration allows a single user access to the webservice service; however, in most organizations, there is an entire web team in charge of managing the service. A single line can also give access to an entire group. These steps will create a web group, add a user to this group, and allow all members of the group to manage the service:

```
# pw groupadd -g 6001 -n webteam
```

Using the same pw(8) command, the user is added to the webteam group:

```
# pw groupmod -m user1 -n webteam
```

Finally, this line in /usr/local/etc/sudoers allows any member of the webteam group to manage *webservice*:

```
%webteam   ALL=(ALL)       /usr/sbin/service webservice *
```

Unlike su(1), sudo(8) only requires the end user password. This avoids sharing passwords, which is a poor practice.

Users permitted to run applications with sudo(8) only enter their own passwords. This is more secure and gives better control than su(1), where the `root` password is entered and the user acquires all `root` permissions.



Most organizations are moving or have moved toward a two factor authentication model. In these cases, the user may not have a password to enter.

sudo(8) can be configured to permit two factor authentication model by using the `NOPASSWD` variable. Adding it to the configuration above will allow all members of the *webteam* group to manage the service without the password requirement:

```
%webteam   ALL=(ALL)       NOPASSWD: /usr/sbin/service webservice *
```

16.3.5. Shared Administration with Doas

doas(1) is a command-line utility ported from OpenBSD. It serves as an alternative to the widely used sudo(8) command in Unix-like systems.

With doas, users can execute commands with elevated privileges, typically as the root user, while maintaining a simplified and security-conscious approach. Unlike sudo(8), doas emphasizes simplicity and minimalism, focusing on streamlined privilege delegation without an overwhelming array of configuration options.

Execute the following command to install it:

```
# pkg install doas
```

After the installation /usr/local/etc/doas.conf must be configured to grant access for users for specific commands, or roles.

The simplest entry could be the following, which grants the user `local_user` with `root` permissions without asking for its password when executing the doas command.

```
permit nopass local_user as root
```

After the installation and configuration of the `doas` utility, a command can now be executed with enhanced privileges, for example:

```
$ doas vi /etc/rc.conf
```

For more configuration examples, please read doas.conf(5).

16.4. Intrusion Detection System (IDS)

Verification of system files and binaries is important because it provides the system administration and security teams information about system changes. A software application that monitors the system for changes is called an Intrusion Detection System (IDS).

FreeBSD provides native support for a basic IDS system called mtree(8). While the nightly security emails will notify an administrator of changes, the information is stored locally and there is a chance that a malicious user could modify this information in order to hide their changes to the system. As such, it is recommended to create a separate set of binary signatures and store them on a read-only, root-owned directory or, preferably, on a removable USB disk or remote server.

It is also recommended to run `freebsd-update IDS` after each update.

16.4.1. Generating the Specification File

The built-in mtree(8) utility can be used to generate a specification of the contents of a directory. A seed, or a numeric constant, is used to generate the specification and is required to check that the specification has not changed. This makes it possible to determine if a file or binary has been modified. Since the seed value is unknown by an attacker, faking or checking the checksum values of files will be difficult to impossible.



It is recommended to create specifications for the directories which contain binaries and configuration files, as well as any directories containing sensitive data. Typically, specifications are created for /bin, /sbin, /usr/bin, /usr/sbin, /usr/local/bin, /etc, and /usr/local/etc.

The following example generates a set of `sha512` hashes, one for each system binary in /bin, and saves those values to a hidden file in user's home directory, /home/user/.bin_chksum_mtree:

```
# mtree -s 123456789 -c -K cksum,sha512 -p /bin > /home/user/.bin_chksum_mtree
```

The output should be similar to the following:

```
mtree: /bin checksum: 3427012225
```



The `123456789` value represents the seed, and should be chosen randomly. This value should be remembered, **but not shared**.

It is important to keep the seed value and the checksum output hidden from malicious users.

16.4.2. The Specification File Structure

The mtree format is a textual format that describes a collection of filesystem objects. Such files are typically used to create or verify directory hierarchies.

An mtree file consists of a series of lines, each providing information about a single filesystem object. Leading whitespace is always ignored.

The specification file created above will be used to explain the format and content:

```
#          user: root ①
#       machine: machinename ②
#          tree: /bin ③
#          date: Thu Aug  24 21:58:37 2023 ④

# .
/set type=file uid=0 gid=0 mode=0555 nlink=1 flags=uarch ⑤
.               type=dir mode=0755 nlink=2 time=1681388848.239523000 ⑥
    \133        nlink=2 size=12520 time=1685991378.688509000 \
                cksum=520880818 \

sha512=5c1374ce0e2ba1b3bc5a41b23f4bbdc1ec89ae82fa01237f376a5eeef41822e68f1d8f75ec46b7b
ceb65396c122a9d837d692740fdebdcc376a05275adbd3471
    cat         size=14600 time=1685991378.694601000 cksum=3672531848 \ ⑦

sha512=b30b96d155fdc4795432b523989a6581d71cdf69ba5f0ccb45d9b9e354b55a665899b16aee21982
fffe20c4680d11da4e3ed9611232a775c69f926e5385d53a2
    chflags     size=8920 time=1685991378.700385000 cksum=1629328991 \

sha512=289a088cbbcbeb436dd9c1f74521a89b66643976abda696b99b9cc1fbfe8b76107c5b54d4a6a9b6
5332386ada73fc1bbb10e43c4e3065fa2161e7be269eaf86a
    chio        size=20720 time=1685991378.706095000 cksum=1948751604 \

sha512=46f58277ff16c3495ea51e74129c73617f31351e250315c2b878a88708c2b8a7bb060e2dc8ff92f
606450dbc7dd2816da4853e465ec61ee411723e8bf52709ee
    chmod       size=9616 time=1685991378.712546000 cksum=4244658911 \

sha512=1769313ce08cba84ecdc2b9c07ef86d2b70a4206420dd71343867be7ab59659956f6f5a458c64e2
531a1c736277a8e419c633a31a8d3c7ccc43e99dd4d71d630
```

① User who created the specification.

② Machine's hostname.

③ Directory path.

④ The Date and time when the specification was created.

⑤ `/set` special commands, defines some settings obtained from the files analyzed.

⑥ Refers to the parsed directory and indicates things like what type it is, its mode, the number of hard links, and the time in UNIX format since it was modified.

⑦ Refers to the file and shows the size, time and a list of hashes to verify the integrity.

16.4.3. Verify the Specification file

To verify that the binary signatures have not changed, compare the current contents of the

directory to the previously generated specification, and save the results to a file.

This command requires the seed that was used to generate the original specification:

```
# mtree -s 123456789 -p /bin < /home/user/.bin_chksum_mtree >>
/home/user/.bin_chksum_output
```

This should produce the same checksum for /bin that was produced when the specification was created. If no changes have occurred to the binaries in this directory, the /home/user/.bin_chksum_output output file will be empty.

To simulate a change, change the date on /bin/cat using touch(1) and run the verification command again:

```
# touch /bin/cat
```

Run the verification command again:

```
# mtree -s 123456789 -p /bin < /home/user/.bin_chksum_mtree >>
/home/user/.bin_chksum_output
```

And then check the content of the output file:

```
# cat /root/.bin_chksum_output
```

The output should be similar to the following:

```
cat:    modification time (Fri Aug 25 13:30:17 2023, Fri Aug 25 13:34:20 2023)
```



This is just an example of what would be displayed when executing the command, to show the changes that would occur in the metadata.

16.5. Secure levels

securelevel is a security mechanism implemented in the kernel. When the securelevel is positive, the kernel restricts certain tasks; not even the superuser (root) is allowed to do them.

The securelevel mechanism limits the ability to:

- Unset certain file flags, such as `schg` (the system immutable flag).
- Write to kernel memory via /dev/mem and /dev/kmem.
- Load kernel modules.

- Alter firewall rules.

16.5.1. Secure Levels Definitions

The kernel runs with five different security levels. Any super-user process can raise the level, but no process can lower it.

The security definitions are:

-1

Permanently insecure mode - always run the system in insecure mode. This is the default initial value.

0

Insecure mode - immutable and append-only flags may be turned off. All devices may be read or written subject to their permissions.

1

Secure mode - the system immutable and system append-only flags may not be turned off; disks for mounted file systems, /dev/mem and /dev/kmem may not be opened for writing; /dev/io (if the platform has it) may not be opened at all; kernel modules (see kld(4)) may not be loaded or unloaded. The kernel debugger may not be entered using the debug.kdb.enter sysctl. A panic or trap cannot be forced using the debug.kdb.panic, debug.kdb.panic_str and other sysctl's.

2

Highly secure mode - same as secure mode, plus disks may not be opened for writing (except by mount(2)) whether mounted or not. This level precludes tampering with file systems by unmounting them, but also inhibits running newfs(8) while the system is multiuser.

3

Network secure mode - same as highly secure mode, plus IP packet filter rules (see ipfw(8), ipfirewall(4) and pfctl(8)) cannot be changed and dummynet(4) or pf(4) configuration cannot be adjusted.



In summary, the key difference between `Permanently Insecure Mode` and `Insecure Mode` in FreeBSD secure levels is the degree of security they provide. `Permanently Insecure Mode` completely lifts all security restrictions, while `Insecure Mode` relaxes some restrictions but still maintains a level of control and security.

16.5.2. Modify Secure Levels

In order to change the securelevel of the system it is necessary to activate `kern_securelevel_enable` by executing the following command:

```
# sysrc kern_securelevel_enable="YES"
```

And set the value of `kern_securelevel` to the desired security level:

```
# sysrc kern_securelevel=2
```

To check the status of the securelevel on a running system execute the following command:

```
# sysctl -n kern.securelevel
```

The output contains the current value of the securelevel. If it is greater than 0, at least some of the securelevel's protections are enabled.

16.6. File flags

File flags allow users to attach additional metadata or attributes to files and directories beyond basic permissions and ownership. These flags provide a way to control various behaviors and properties of files without needing to resort to creating special directories or using extended attributes.

File flags can be used to achieve different goals, such as preventing file deletion, making files append-only, synchronizing file updates, and more. Some commonly used file flags in FreeBSD include the "immutable" flag, which prevents modification or deletion of a file, and the "append-only" flag, which allows only data to be added to the end of a file but not modified or removed.

These flags can be managed using the chflags(1) command in FreeBSD, providing administrators and users with greater control over the behavior and characteristics of their files and directories. It is important to note that file flags are typically managed by root or users with appropriate privileges, as they can influence how files are accessed and manipulated. Some flags are available for the use of the file's owner, as described in chflags(1).

16.6.1. Work with File Flags

In this example, a file named ~/important.txt in user's home directory want to be protected against deletions.

Execute the following command to set the `schg` file flag:

```
# chflags schg ~/important.txt
```

When any user, including the `root` user, tries to delete the file, the system will display the message:

```
rm: important.txt: Operation not permitted
```

To delete the file, it will be necessary to delete the file flags of that file by executing the following command:

```
# chflags noschg ~/important.txt
```

A list of supported file flags and their functionality can be found in chflags(1).

16.7. OpenSSH

OpenSSH is a set of network connectivity tools used to provide secure access to remote machines. Additionally, TCP/IP connections can be tunneled or forwarded securely through SSH connections. OpenSSH encrypts all traffic to eliminate eavesdropping, connection hijacking, and other network-level attacks.

OpenSSH is maintained by the OpenBSD project and is installed by default in FreeBSD.

When data is sent over the network in an unencrypted form, network sniffers anywhere in between the client and server can steal user/password information or data transferred during the session. OpenSSH offers a variety of authentication and encryption methods to prevent this from happening.

More information about OpenSSH is available in the web page.

This section provides an overview of the built-in client utilities to securely access other systems and securely transfer files from a FreeBSD system. It then describes how to configure a SSH server on a FreeBSD system.



As stated, this chapter will cover the base system version of OpenSSH. A version of OpenSSH is also available in the security/openssh-portable, which provides additional configuration options and is updated with new features more regularly.

16.7.1. Using the SSH Client Utilities

To log into a SSH server, use ssh(1) and specify a username that exists on that server and the IP address or hostname of the server. If this is the first time a connection has been made to the specified server, the user will be prompted to first verify the server's fingerprint:

```
# ssh user@example.com
```

The output should be similar to the following:

```
The authenticity of host 'example.com (10.0.0.1)' can't be established.
ECDSA key fingerprint is 25:cc:73:b5:b3:96:75:3d:56:19:49:d2:5c:1f:91:3b.
Are you sure you want to continue connecting (yes/no)? yes
Permanently added 'example.com' (ECDSA) to the list of known hosts.
Password for user@example.com: user_password
```

SSH utilizes a key fingerprint system to verify the authenticity of the server when the client connects. When the user accepts the key's fingerprint by typing `yes` when connecting for the first

time, a copy of the key is saved to ~/.ssh/known_hosts in the user's home directory. Future attempts to login are verified against the saved key and ssh(1) will display an alert if the server's key does not match the saved key. If this occurs, the user should first verify why the key has changed before continuing with the connection.



How to perform this check is outside the scope of this chapter.

Use scp(1) to securely copy a file to or from a remote machine.

This example copies `COPYRIGHT` on the remote system to a file of the same name in the current directory of the local system:

```
# scp user@example.com:/COPYRIGHT COPYRIGHT
```

The output should be similar to the following:

```
Password for user@example.com: *******
COPYRIGHT            100% |*****************************|  4735
```

Since the fingerprint was already verified for this host, the server's key is automatically checked before prompting for the user's password.

The arguments passed to scp(1) are similar to cp(1). The file or files to copy is the first argument and the destination to copy to is the second. Since the file is fetched over the network, one or more of the file arguments takes the form `user@host:<path_to_remote_file>`. Be aware when copying directories recursively that scp(1) uses `-r`, whereas cp(1) uses `-R`.

To open an interactive session for copying files, use sftp(1).

Refer to sftp(1) for a list of available commands while in an sftp(1) session.

16.7.2. Key-based Authentication

Instead of using passwords, a client can be configured to connect to the remote machine using keys. For security reasons, this is the preferred method.

ssh-keygen(1) can be used to generate the authentication keys. To generate a public and private key pair, specify the type of key and follow the prompts. It is recommended to protect the keys with a memorable, but hard to guess passphrase.

```
% ssh-keygen -t rsa -b 4096
```

The output should be similar to the following:

```
Generating public/private rsa key pair.
Enter file in which to save the key (/home/user/.ssh/id_rsa):
```

```
Created directory '/home/user/.ssh/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/user/.ssh/id_rsa.
Your public key has been saved in /home/user/.ssh/id_rsa.pub.
The key fingerprint is:
SHA256:54Xm9Uvtv6H4NOo6yjP/YCfODryvUU7yWHzMqeXwhq8 user@host.example.com
The key's randomart image is:
+---[RSA 2048]----+
|                 |
|                 |
|                 |
|        . o..    |
|       .S*+*o    |
|      . O=Oo . . |
|       = Oo= oo..|
|      .oB.* +.oo.|
|       =OE**.o..=|
+----[SHA256]-----+
```

The private key is stored in ~/.ssh/id_rsa and the public key is stored in ~/.ssh/id_rsa.pub. The *public* key must be copied to ~/.ssh/authorized_keys on the remote machine for key-based authentication to work.



Utilizing a passphrase for OpenSSH keys is a key security practice, providing an extra layer of protection against unauthorized access and enhancing overall cybersecurity.

In case of loss or theft, this adds another layer of security.

16.7.3. SSH Tunneling

OpenSSH has the ability to create a tunnel to encapsulate another protocol in an encrypted session.

The following command tells ssh(1) to create a tunnel:

```
% ssh -D 8080 user@example.com
```

This example uses the following options:

-D

Specifies a local "dynamic" application-level port forwarding.

user@foo.example.com

The login name to use on the specified remote SSH server.

An SSH tunnel works by creating a listen socket on `localhost` on the specified `localport`.

This method can be used to wrap any number of insecure TCP protocols such as SMTP, POP3, and FTP.

16.7.4. Enabling the SSH Server

In addition to providing built-in SSH client utilities, a FreeBSD system can be configured as an SSH server, accepting connections from other SSH clients.



As stated, this chapter will cover the base system version of OpenSSH. Please **not** confuse with security/openssh-portable, the version of OpenSSH that ships with the FreeBSD ports.

In order to have the SSH Server enabled across reboots execute the following command:

```
# sysrc sshd_enable="YES"
```

Then execute the following command to enable the service:

```
# service sshd start
```

The first time sshd starts on a FreeBSD system, the system's host keys will be automatically created and the fingerprint will be displayed on the console. Provide users with the fingerprint so that they can verify it the first time they connect to the server.

Refer to sshd(8) for the list of available options when starting sshd and a complete discussion about authentication, the login process, and the various configuration files.

At this point, the sshd should be available to all users with a username and password on the system.

16.7.5. Configuring publickey auth method

Configuring OpenSSH to use public key authentication enhances security by leveraging asymmetric cryptography for authentication. This method eliminates password-related risks, such as weak passwords or interception during transmission, while thwarting various password-based attacks. However, it's vital to ensure the private keys are well-protected to prevent unauthorized access.

The first step will be to configure sshd(8) to use the required authentication method.

Edit /etc/ssh/sshd_config and uncomment the following configuration:

```
PubkeyAuthentication yes
```

Once the configuration is done, the users will have to send the system administrator their **public key** and these keys will be added in .ssh/authorized_keys. The process for generating the keys is described in Key-based Authentication.

Then restart the server executing the following command:

```
# service sshd reload
```

It is strongly recommended to follow the security improvements indicated in SSH Server Security Options.

16.7.6. SSH Server Security Options

While sshd is the most widely used remote administration facility for FreeBSD, brute force and drive by attacks are common to any system exposed to public networks.

Several additional parameters are available to prevent the success of these attacks and will be described in this section. All configurations will be done in /etc/ssh/sshd_config



Do not confuse /etc/ssh/sshd_config with /etc/ssh/ssh_config (note the extra `d` in the first filename). The first file configures the server and the second file configures the client. Refer to ssh_config(5) for a listing of the available client settings.

By default, authentication can be done with both pubkey and password. To allow **only** pubkey authentication, **which is strongly recommended**, change the variable:

```
PasswordAuthentication no
```

It is a good idea to limit which users can log into the SSH server and from where using the `AllowUsers` keyword in the OpenSSH server configuration file. For example, to only allow `user` to log in from `192.168.1.32`, add this line to /etc/ssh/sshd_config:

```
AllowUsers user@192.168.1.32
```

To allow `user` to log in from anywhere, list that user without specifying an IP address:

```
AllowUsers user
```

Multiple users should be listed on the same line, like so:

```
AllowUsers root@192.168.1.32 user
```

After making all the changes, and before restarting the service, it is recommended to verify that the configuration made is correct by executing the following command:

```
# sshd -t
```

If the configuration file is correct, no output will be shown. In case the configuration file is incorrect, it will show something like this:

```
/etc/ssh/sshd_config: line 3: Bad configuration option: sdadasdasdasads
/etc/ssh/sshd_config: terminating, 1 bad configuration options
```

After making the changes and checking that the configuration file is correct, tell sshd to reload its configuration file by running:

```
# service sshd reload
```

16.8. OpenSSL

OpenSSL is a cryptography toolkit implementing the Secure Sockets Layer (SSL) and Transport Layer Security (TLS) network protocols and many cryptography routines.

The openssl program is a command line tool for using the various cryptography functions of OpenSSL's crypto library from the shell. It can be used for

- Creation and management of private keys, public keys and parameters
- Public key cryptographic operations
- Creation of X.509 certificates, CSRs and CRLs
- Calculation of Message Digests
- Encryption and Decryption with Ciphers
- SSL/TLS Client and Server Tests
- Handling of S/MIME signed or encrypted mail
- Time Stamp requests, generation and verification
- Benchmarking the crypto routines

For more information about OpenSSL, read the free OpenSSL Cookbook.

16.8.1. Generating Certificates

OpenSSL supports the generation of certificates both to be validated by a CA and for own use.

Run the command openssl(1) to generate a valid certificate for a CA with the following arguments. This command will create two files in the current directory. The certificate request, req.pem, can be sent to a CA which, will validate the entered credentials, sign the request, and return the signed certificate. The second file, cert.key, is the private key for the certificate and should be stored in a secure location. If this falls in the hands of others, it can be used to impersonate the user or the server.

Execute the following command to generate the certificate:

```
# openssl req -new -nodes -out req.pem -keyout cert.key -sha3-512 -newkey rsa:4096
```

The output should be similar to the following:

```
Generating a RSA private key
..................................................................................
.........................................+++++
.....................................+++++
writing new private key to 'cert.key'
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:ES
State or Province Name (full name) [Some-State]:Valencian Community
Locality Name (eg, city) []:Valencia
Organization Name (eg, company) [Internet Widgits Pty Ltd]:My Company
Organizational Unit Name (eg, section) []:Systems Administrator
Common Name (e.g. server FQDN or YOUR name) []:localhost.example.org
Email Address []:user@FreeBSD.org

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:123456789
An optional company name []:Another name
```

Alternately, if a signature from a CA is not required, a self-signed certificate can be created. This will create two new files in the current directory: a private key file cert.key, and the certificate itself, cert.crt. These should be placed in a directory, preferably under /etc/ssl/, which is readable only by `root`. Permissions of `0700` are appropriate for these files and can be set using `chmod`.

Execute the following command to generate the certificate:

```
# openssl req -new -x509 -days 365 -sha3-512 -keyout /etc/ssl/private/cert.key -out
/etc/ssl/certs/cert.crt
```

The output should be similar to the following:

```
Generating a RSA private key
.....................................+++++
...........+++++
writing new private key to '/etc/ssl/private/cert.key'
Enter PEM pass phrase:
```

```
Verifying - Enter PEM pass phrase:
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:ES
State or Province Name (full name) [Some-State]:Valencian Community
Locality Name (eg, city) []:Valencia
Organization Name (eg, company) [Internet Widgits Pty Ltd]:My Company
Organizational Unit Name (eg, section) []:Systems Administrator
Common Name (e.g. server FQDN or YOUR name) []:localhost.example.org
Email Address []:user@FreeBSD.org
```

16.8.2. Configuring the FIPS Provider

With the import of OpenSSL 3 into the base system (on FreeBSD 14 and later), its new concept of provider modules was introduced in the system. Besides the default provider module built-in to the library, the *legacy* module implements the now optional deprecated cryptography algorithms, while the *fips* module restricts the OpenSSL implementation to the cryptography algorithms present in the FIPS set of standards. This part of OpenSSL receives particular care, including a list of relevant security issues, and is subject to the FIPS 140 validation process on a regular basis. The list of FIPS validated versions is also available. This allows users to ensure FIPS compliance in their use of OpenSSL.

Importantly, the fips_module(7) is protected by an additional security measure, preventing its use without passing an integrity check. This check can be setup by the local system administrator, allowing every user of OpenSSL 3 to load this module. When not configured correctly, the FIPS module is expected to fail as follows:

```
# echo test | openssl aes-128-cbc -a -provider fips -pbkdf2
```

The output should be similar to the following:

```
aes-128-cbc: unable to load provider fips
Hint: use -provider-path option or OPENSSL_MODULES environment variable.
00206124D94D0000:error:1C8000D5:Provider routines:SELF_TEST_post:missing config
data:crypto/openssl/providers/fips/self_test.c:275:
00206124D94D0000:error:1C8000E0:Provider routines:ossl_set_error_state:fips module
entering error state:crypto/openssl/providers/fips/self_test.c:373:
00206124D94D0000:error:1C8000D8:Provider routines:OSSL_provider_init_int:self test
post failure:crypto/openssl/providers/fips/fipsprov.c:707:
00206124D94D0000:error:078C0105:common libcrypto routines:provider_init:init
fail:crypto/openssl/crypto/provider_core.c:932:name=fips
```

The check can be configured through the creation of a file in /etc/ssl/fipsmodule.cnf, which will then be referenced in OpenSSL's main configuration file /etc/ssl/openssl.cnf. OpenSSL provides the openssl-fipsinstall(1) utility to help with this process, which can be used as follows:

```
# openssl fipsinstall -module /usr/lib/ossl-modules/fips.so -out
/etc/ssl/fipsmodule.cnf
```

The output should be similar to the following:

```
INSTALL PASSED
```

The /etc/ssl/openssl.cnf should then be modified, in order to:

- Include the /etc/ssl/fipsmodule.cnf file generated above,
- Expose the FIPS module for possible use,
- And explicitly activate the default module.

```
[...]
# For FIPS
# Optionally include a file that is generated by the OpenSSL fipsinstall
# application. This file contains configuration data required by the OpenSSL
# fips provider. It contains a named section e.g. [fips_sect] which is
# referenced from the [provider_sect] below.
# Refer to the OpenSSL security policy for more information.
.include /etc/ssl/fipsmodule.cnf

[...]

# List of providers to load
[provider_sect]
default = default_sect
# The fips section name should match the section name inside the
# included fipsmodule.cnf.
fips = fips_sect

# If no providers are activated explicitly, the default one is activated implicitly.
# See man 7 OSSL_PROVIDER-default for more details.
#
# If you add a section explicitly activating any other provider(s), you most
# probably need to explicitly activate the default provider, otherwise it
# becomes unavailable in openssl.  As a consequence applications depending on
# OpenSSL may not work correctly which could lead to significant system
# problems including inability to remotely access the system.
[default_sect]
activate = 1
```

With this done, it should be possible to confirm that the FIPS module is effectively available and working:

```
# echo test | openssl aes-128-cbc -a -provider fips -pbkdf2
```

The output should be similar to the following:

```
enter AES-128-CBC encryption password:
Verifying - enter AES-128-CBC encryption password:
U2FsdGVkX18idooW6e3LqWeeiKP76kufcOUClh57j8U=
```

This procedure has to be repeated every time the FIPS module is modified, e.g., after performing system updates, or after applying security fixes affecting OpenSSL in the base system.

16.9. Kerberos

Kerberos is a network authentication protocol which was originally created by the Massachusetts Institute of Technology (MIT) as a way to securely provide authentication across a potentially hostile network. The Kerberos protocol uses strong cryptography so that both a client and server can prove their identity without sending any unencrypted secrets over the network. Kerberos can be described as an identity-verifying proxy system and as a trusted third-party authentication system. After a user authenticates with Kerberos, their communications can be encrypted to assure privacy and data integrity.

The only function of Kerberos is to provide the secure authentication of users and servers on the network. It does not provide authorization or auditing functions. It is recommended that Kerberos be used with other security methods which provide authorization and audit services.

The current version of the protocol is version 5, described in RFC 4120. Several free implementations of this protocol are available, covering a wide range of operating systems. MIT continues to develop their Kerberos package. It is commonly used in the US as a cryptography product, and has historically been subject to US export regulations. In FreeBSD, MITKerberos is available as the security/krb5 package or port. The Heimdal Kerberos implementation was explicitly developed outside of the US to avoid export regulations. The Heimdal Kerberos distribution is included in the base FreeBSD installation, and another distribution with more configurable options is available as security/heimdal in the Ports Collection.

In Kerberos users and services are identified as "principals" which are contained within an administrative grouping, called a "realm". A typical user principal would be of the form `user@REALM` (realms are traditionally uppercase).

This section provides a guide on how to set up Kerberos using the Heimdal distribution included in FreeBSD.

For purposes of demonstrating a Kerberos installation, the name spaces will be as follows:

- The DNS domain (zone) will be `example.org`.

- The Kerberos realm will be `EXAMPLE.ORG`.



Use real domain names when setting up Kerberos, even if it will run internally. This avoids DNS problems and assures inter-operation with other Kerberos realms.

16.9.1. Setting up a Heimdal KDC

The Key Distribution Center (KDC) is the centralized authentication service that Kerberos provides, the "trusted third party" of the system. It is the computer that issues Kerberos tickets, which are used for clients to authenticate to servers. As the KDC is considered trusted by all other computers in the Kerberos realm, it has heightened security concerns. Direct access to the KDC should be limited.

While running a KDC requires few computing resources, a dedicated machine acting only as a KDC is recommended for security reasons.

To begin, install the security/heimdal package as follows:

```
# pkg install heimdal
```

Next, update /etc/rc.conf using `sysrc` as follows:

```
# sysrc kdc_enable=yes
# sysrc kadmind_enable=yes
```

Next, edit /etc/krb5.conf as follows:

```
[libdefaults]
    default_realm = EXAMPLE.ORG
[realms]
    EXAMPLE.ORG = {
    kdc = kerberos.example.org
    admin_server = kerberos.example.org
    }
[domain_realm]
    .example.org = EXAMPLE.ORG
```

In this example, the KDC will use the fully-qualified hostname `kerberos.example.org`. The hostname of the KDC must be resolvable in the DNS.

Kerberos can also use the DNS to locate KDCs, instead of a `[realms]` section in /etc/krb5.conf. For large organizations that have their own DNS servers, the above example could be trimmed to:

```
[libdefaults]
      default_realm = EXAMPLE.ORG
[domain_realm]
```